

# Contents

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>1. Objective</b>                                          | <b>3</b>  |
| <b>2. Reference Dataset</b>                                  | <b>3</b>  |
| <b>3. Approach</b>                                           | <b>3</b>  |
| <b>4. Architecture</b>                                       | <b>3</b>  |
| 4.1 Structure of app.py                                      | 4         |
| 4.2 Model architecture                                       | 4         |
| 4.3 Preprocessing and test-time augmentation                 | 5         |
| 4.4 Augmentation recipe                                      | 5         |
| 4.5 Live evaluation and caching                              | 6         |
| 4.6 Running the application                                  | 6         |
| 4.7 Testing and issues resolved                              | 6         |
| <b>5. Source Code</b>                                        | <b>7</b>  |
| <b>6. Screenshots of the Interface</b>                       | <b>16</b> |
| 6.1 Home — Overview                                          | 16        |
| 6.2 Try It Live — Sample Gallery                             | 17        |
| 6.3 Try It Live — Sample Selection and Prediction Result     | 18        |
| 6.4 Try It Live — Drawing Canvas and Prediction              | 19        |
| 6.5 Try It Live — Photo Upload Result and Session History    | 20        |
| 6.6 Model Dashboard — Clean Validation Set                   | 21        |
| 6.7 Model Dashboard — Messy / Augmented Set                  | 22        |
| 6.8 Model Dashboard — Per-Class Table                        | 23        |
| 6.9 Model Dashboard — Calibration and Misclassifications     | 24        |
| 6.10 Dataset Explorer — Class Balance and Mean Digits        | 25        |
| 6.11 Dataset Explorer — Raw Browser and Augmentation Preview | 26        |
| 6.12 Documentation Page (1 of 2)                             | 27        |
| 6.13 Documentation Page (2 of 2)                             | 28        |
| <b>7. Results</b>                                            | <b>29</b> |
| 7.1 Per-class precision, recall and F1                       | 29        |
| 7.2 Most frequent confusions                                 | 30        |
| 7.3 Reliability data (messy set)                             | 30        |
| <b>8. Observations</b>                                       | <b>30</b> |
| <b>9. Known Limitations</b>                                  | <b>31</b> |
| <b>10. Conclusion</b>                                        | <b>31</b> |

## 1. Objective

To develop a professional deep-learning application with multiple pages and interactive visualizations: extend a single-page digit classifier into a true multi-page Streamlit application with a live classifier, a dedicated model-performance dashboard, a dataset explorer and in-app documentation, all sharing one model, one preprocessing pipeline and one design system.

## 2. Reference Dataset

The UCI ML handwritten digits dataset (1,797 images, via `sklearn.datasets.load_digits`): 8×8 grayscale images with 4-bit intensities (0–16) across 10 classes, with 174–183 images per digit. The application uses it as the browsable sample gallery, as the source of the held-out validation split used for evaluation, and as the base images for the augmentation preview. The frozen weights are loaded from `cnn_digits_model.pkl`; this application is an inference and analysis layer, not a retrain.

## 3. Approach

1. **Create a multi-page architecture** — five `st.Page` objects (Home, Try It Live, Model Dashboard, Dataset Explorer, Documentation) registered with `st.navigation` in one file, giving native sidebar navigation (Figure 1).
2. **Add dashboards and visualizations** — a Model Dashboard page (confusion matrices, per-class metrics, calibration, misclassification gallery) and a Dataset Explorer page (class balance, mean-digit images, live augmentation preview), drawn with Altair (Figures 6–11).
3. **Display model performance metrics** — every number on the dashboard is computed live by `run_evaluation` directly from the frozen model: accuracy, macro-F1, expected calibration error, precision/recall/F1 per class and the most frequent confusions (Figures 6–9).
4. **Integrate charts and reports** — interactive Altair tooltips, CSV downloads of the per-class report and of the prediction history, and an in-app Documentation page that doubles as a living project report (Figures 5, 8, 12, 13).
5. **Improve navigation and user experience** — a consistent hero banner and metric cards, a sidebar model-status badge, numbered step guidance, tabs, spinners, empty-state messages and a session history (Figures 1, 2, 5).

## 4. Architecture

Five pages, one file, one shared model. `app.py` contains the theme, the NumPy CNN, preprocessing, augmentation, the cached live evaluation and every page, so each page reuses exactly the same model and augmentation code instead of carrying its own copy. The project consists of four files.

| File                              | Role                                                                                                                                                                       |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>app.py</code>               | The entire application (862 lines, 42.4 KB): theme, model, preprocessing, augmentation, live evaluation and five pages. Entry point ( <code>streamlit run app.py</code> ). |
| <code>cnn_digits_model.pkl</code> | Trained weights plus metadata (training date, epochs, augmentation factor, recorded accuracies, weights hash); 61.2 KB. Must sit next to <code>app.py</code> .             |
| <code>requirements.txt</code>     | Python dependencies (listed in Section 5.13).                                                                                                                              |
| <code>README.md</code>            | Project overview, setup steps, known limitations and deployment notes.                                                                                                     |

## 4.1 Structure of app.py

| Lines   | Section of <code>app.py</code>      | What it contains                                                                                                                                 |
|---------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-42    | Header and imports                  | Module docstring, imports, model path and class-name constants                                                                                   |
| 43-114  | 1. Theme                            | Shared CSS, <code>hero()</code> , <code>metric_row()</code> , numbered <code>steps()</code> , footer and confidence-colour logic                 |
| 115-183 | 2. Model                            | NumPy <code>im2col</code> convolution, max-pool, softmax and the inference-only <code>DigitCNN</code> class (weights, metadata, parameter table) |
| 184-260 | 3. Preprocessing                    | Crop-to-ink, per-axis scaling, centre-by-mass, dilate/erode and <code>predict_with_tta</code>                                                    |
| 261-309 | 4. Augmentation                     | <code>elastic_warp</code> and <code>augment_one</code> : the training-time distortion recipe, reused for evaluation and the explorer preview     |
| 310-378 | 5. Cached loaders + live evaluation | <code>get_model</code> , <code>get_dataset</code> and the cached <code>run_evaluation</code> (metrics, calibration, ECE, errors, class means)    |
| 379-835 | 6. Pages                            | <code>sidebar_badge</code> and the five page functions: Home, Try It Live, Model Dashboard, Dataset Explorer, Documentation                      |
| 836-862 | 7. Entry point                      | <code>main()</code> : page config, model-file check, <code>st.Page</code> objects and <code>st.navigation</code>                                 |

## 4.2 Model architecture

A small convolutional network implemented in plain NumPy: two convolution blocks followed by a two-layer classifier. Convolutions use 3×3 kernels with 'same' padding via an `im2col` implementation. The model has 7,754 trainable parameters. It was trained from scratch with hand-derived backpropagation for 24 epochs with 18 augmented variants per training image; the application itself ships only the inference path.

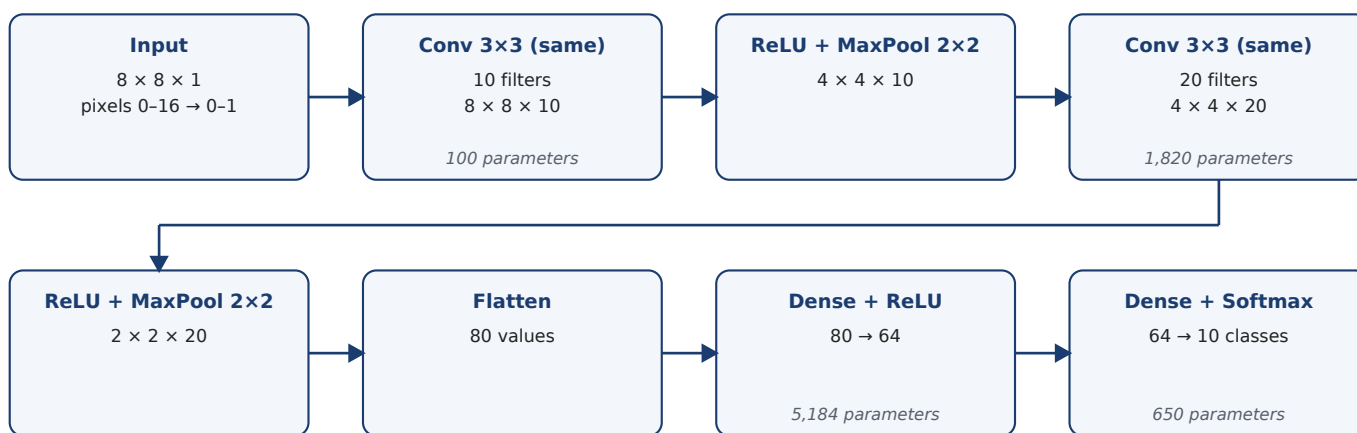


Diagram 1: Network layers with activation shapes and parameter counts (total 7,754).

| Parameter    | Shape                            | Count        |
|--------------|----------------------------------|--------------|
| conv1_W      | $3 \times 3 \times 1 \times 10$  | 90           |
| conv1_b      | 10                               | 10           |
| conv2_W      | $3 \times 3 \times 10 \times 20$ | 1,800        |
| conv2_b      | 20                               | 20           |
| fc1_W        | $80 \times 64$                   | 5,120        |
| fc1_b        | 64                               | 64           |
| fc2_W        | $64 \times 10$                   | 640          |
| fc2_b        | 10                               | 10           |
| <b>Total</b> |                                  | <b>7,754</b> |

### 4.3 Preprocessing and test-time augmentation

Every input, whether a sample, a canvas drawing or an uploaded photo, passes through the same pipeline, so live input is shaped like the data the model was trained on. The prediction shown to the user is the average over eleven slightly different versions of the input.

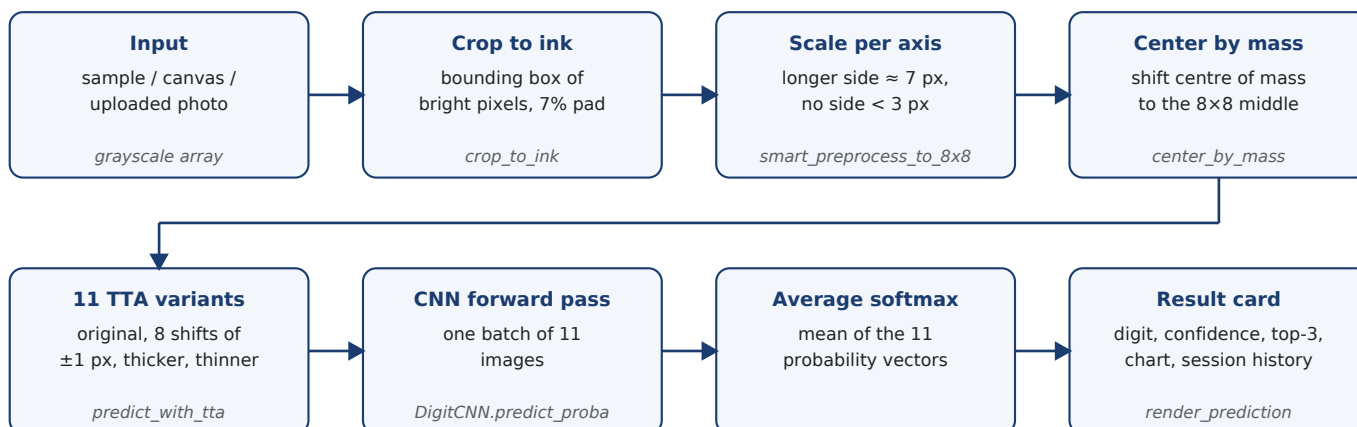


Diagram 2: Live prediction pipeline. Function names refer to `app.py`.

### 4.4 Augmentation recipe

`augment_one` upsamples an  $8 \times 8$  digit to  $160 \times 160$  and applies the distortions below before re-running the preprocessing pipeline. The same function drives the messy evaluation set and the live preview in the Dataset Explorer.

| Step          | Setting in the code                                                                                                            |
|---------------|--------------------------------------------------------------------------------------------------------------------------------|
| Rotation      | uniform in $-22^\circ$ to $+22^\circ$                                                                                          |
| Scaling       | independent per axis, 0.65–1.4; with 30 % probability an extreme stretch/squish instead (one axis 0.45–0.7, the other 1.3–1.7) |
| Translation   | up to $\pm 1/7$ of the canvas (about $\pm 22$ px at 160 px) on each axis                                                       |
| Elastic warp  | applied with 60 % probability: two sine displacement fields with random frequency and phase                                    |
| Stroke weight | 55 % probability of 1–2 dilations (thicker); otherwise about 16 % probability of one erosion (thinner)                         |
| Noise         | Gaussian noise, $\sigma = 7$ on a 0–255 scale                                                                                  |

## 4.5 Live evaluation and caching

`run_evaluation` rebuilds the validation split, scores the frozen model on the clean images and on freshly augmented variants, and derives every number the dashboard shows. The result is cached, so the cost is paid once per server process (about 8 seconds on the machine used for this report).

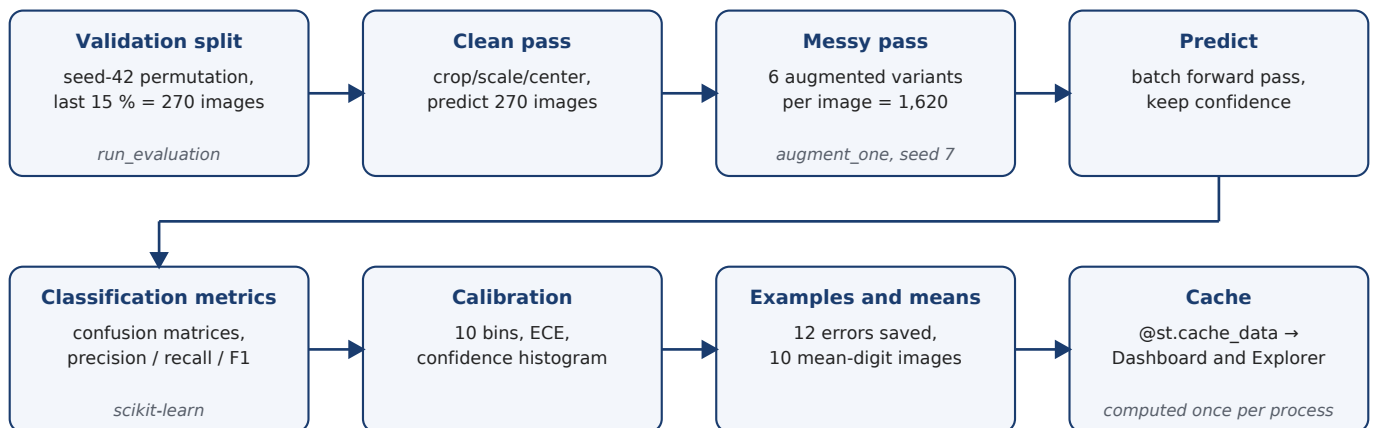


Diagram 3: Evaluation pipeline behind the Model Dashboard and the Dataset Explorer.

## 4.6 Running the application

- Place `app.py`, `cnn_digits_model.pkl` and `requirements.txt` in one folder.
- Install dependencies: `pip install -r requirements.txt`.
- Start the app: `streamlit run app.py`; it opens at `http://localhost:8501`.
- After replacing the model file, restart the app so the cached model and metrics refresh; the sidebar fingerprint confirms which weights are loaded.

`requirements.txt` specifies `streamlit>=1.53`. The code uses `width="stretch"` on several widgets, which raised an error on Streamlit 1.46 and 1.50 in testing and works on 1.52, 1.53 and 1.65, and the drawing-canvas package itself requires 1.53 or newer.

## 4.7 Testing and issues resolved

A final review ran every page end to end and compared each figure with the model's own output. The issues below were fixed in `app.py` and `requirements.txt` and re-verified in the running application; the model and metrics are unchanged.

| Issue found                                                           | Cause                                                                  | Fix and verification                                                                     |
|-----------------------------------------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Session history logged a prediction twice (3 predictions gave 4 rows) | The duplicate check compared only with the last entry.                 | Log once per input per source. Verified: exactly 3 rows (Figure 5); Clear history works. |
| Hero banner clipped under the toolbar                                 | Top padding of 2 rem was below the header height.                      | Padding 4.5 rem (Figure 1).                                                              |
| <code>requirements.txt</code> allowed Streamlit 1.46                  | <code>width="stretch"</code> fails before 1.52; the canvas needs 1.53. | <code>streamlit&gt;=1.53</code> ; 1.46 and 1.50 fail, 1.52, 1.53, 1.65 pass.             |
| Architecture line on Home cut off                                     | One long line in a code box.                                           | <code>wrap_lines=True</code> .                                                           |
| Reliability diagram axes ran -0.1 to 1.1, stray "x" title             | The two chart layers used different axes.                              | Shared fixed 0-1 axes and titles (Figure 9).                                             |
| Small tidy-ups                                                        | Duplicate sidebar divider, unused import, README timing.               | Removed; README updated.                                                                 |

## 5. Source Code

The whole application is one file, `app.py` (862 lines), listed here in the order a reader would naturally open it: header and imports, theme, model, preprocessing, augmentation, cached evaluation, the five pages and the entry point, followed by the dependency list. Line numbers refer to the real file. Lines longer than 132 characters are wrapped and continued with “↳”; 28 emoji and symbol characters that the PDF’s monospaced font cannot draw are shown as Unicode code points, for example <U+1F522>.

### 5.1 `app.py` — Module header, imports and constants (lines 1-42)

```
1 """
2 app.py - Digit Intelligence Suite
3 =====
4 A professional multi-page deep-learning application in ONE file.
5
6 Run: streamlit run app.py
7 Needs (same folder): cnn_digits_model.pkl
8
9 Pages (sidebar navigation):
10 Home - overview, headline metrics, architecture, quick links
11 Try It Live - sample gallery / draw / upload -> CNN prediction
12 Model Dashboard - confusion matrices, P/R/F1, calibration, errors, report download
13 Dataset Explorer - class balance, mean digits, raw browser, augmentation preview
14 Documentation - objective, approach, deliverables, architecture, limitations
15
16 Everything (model, preprocessing, augmentation, evaluation, theme) lives in
17 this file. Evaluation is computed live from the frozen model and cached, so
18 eval_results.pkl is no longer needed.
19 """
20 import hashlib
21 import pickle
22 from pathlib import Path
23
24 import altair as alt
25 import numpy as np
26 import pandas as pd
27 import streamlit as st
28 from PIL import Image
29 from sklearn.datasets import load_digits
30 from sklearn.metrics import classification_report, confusion_matrix
31
32 try:
33     from streamlit_drawable_canvas import st_canvas
34     CANVAS_AVAILABLE, CANVAS_IMPORT_ERROR = True, None
35 except Exception as e: # version mismatches raise non-ImportErrors too
36     CANVAS_AVAILABLE, CANVAS_IMPORT_ERROR = False, str(e)
37
38 MODEL_PATH = Path(__file__).parent / "cnn_digits_model.pkl"
39 DIGITS = [str(i) for i in range(10)]
40 ACCENT, GOOD, MID, BAD, NAVY = "#FF4B4B", "#21C55D", "#F59E0B", "#EF4444", "#1A3C6E"
41
42
```

### 5.2 `app.py` — Theme: CSS and HTML helpers (lines 43-114)

```
43 # =====
44 # 1. THEME
45 # =====
46 CSS = """
47 <style>
48 @import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=JetBrains+Mono:wght@500;600&display=swap
↳ ');
49 .block-container { padding-top: 4.5rem; padding-bottom: 3rem; max-width: 1300px; }
50 html, body, [class*="css"] { font-family: 'Inter', sans-serif; }
51 .hero { background: linear-gradient(180deg, #17181c 0%, #101114 100%);
52     border: 1px solid rgba(255,255,255,.08); border-top: 3px solid #FF4B4B;
53     border-radius: 10px; padding: 1.5rem 2rem; margin-bottom: 1.4rem; }
54 .hero-eyebrow { display: inline-flex; align-items: center; gap: .5rem;
55     font-family: 'JetBrains Mono', monospace; font-size: .75rem; font-weight: 600;
56     letter-spacing: .14em; text-transform: uppercase; color: #9CA3AF; margin-bottom: .8rem; }
57 .hero-eyebrow:after { content: ''; width: 8px; height: 8px; border-radius: 2px; background: #FF4B4B; }
58 .hero h1 { margin: 0 !important; font-weight: 800 !important; letter-spacing: -.02em; color: #F5F5F7; font-size: 2.1rem !important; }
59 .hero p.hero-sub { margin: .6rem 0 0 0; color: #c9ccd1; font-size: 1.02rem; line-height: 1.55; max-width: 800px; }
60 .metric-row { display: flex; gap: .9rem; flex-wrap: wrap; margin-bottom: 1.4rem; }
61 .metric-card { flex: 1; min-width: 160px; border: 1px solid #e5e7eb; border-radius: 10px;
62     padding: .9rem 1.1rem; background: #f9f9f9; }
63 .mc-label { font-size: .76rem; text-transform: uppercase; letter-spacing: .05em; color: #6b7280; }
64 .mc-value { font-family: 'JetBrains Mono', monospace; font-size: 1.5rem; font-weight: 700; color: #111827; margin-top: .2rem; }
65 .mc-sub { font-size: .78rem; color: #6b7280; margin-top: .1rem; }
66 .result-card { border-radius: 18px; padding: 1.6rem 1.4rem; text-align: center;
67     border: 2px solid var(--card-border); background: var(--card-bg); }
68 .result-digit { font-size: 4.5rem; font-weight: 800; line-height: 1; margin: .2rem 0; }
69 .result-label { font-size: 1.1rem; opacity: .75; text-transform: uppercase; letter-spacing: .06em; }
70 .confidence-pill { display: inline-block; padding: .35rem 1rem; border-radius: 999px;
71     font-weight: 700; font-size: 1.02rem; margin-top: .5rem; }
72 .step-badge { display: inline-flex; align-items: center; justify-content: center; width: 30px; height: 30px;
73     border-radius: 50%; background: #FF4B4B; color: #fff; font-weight: 800; margin-right: 10px; flex-shrink: 0; }
74 .step-row { display: flex; align-items: flex-start; margin-bottom: .7rem; }
75 .step-text { padding-top: 4px; font-size: 1.02rem; }
76 .app-footer { text-align: center; opacity: .55; font-size: .85rem; margin-top: 2rem; }
77 </style>
78 """
79
80
81 def hero(eyebrow, title, subtitle):
82     st.markdown(
83         f'<div class="hero"><span class="hero-eyebrow">{eyebrow}</span>
84         f'<h1>{title}</h1><p class="hero-sub">{subtitle}</p></div>',
85         unsafe_allow_html=True)

```

```

86
87
88 def metric_row(items):
89     cards = "".join(
90         f'<div class="metric-card"><div class="mc-label">{a}</div>'
91         f'<div class="mc-value">{b}</div><div class="mc-sub">{c}</div></div>'
92         for a, b, c in items)
93     st.markdown(f'<div class="metric-row">{cards}</div>', unsafe_allow_html=True)
94
95
96 def steps(*texts):
97     st.markdown("".join(
98         f'<div class="step-row"><div class="step-badge">{i}</div><div class="step-text">{t}</div></div>'
99         for i, t in enumerate(texts, 1)), unsafe_allow_html=True)
100
101
102 def footer():
103     st.markdown("<div class='app-footer'>Digit Intelligence Suite &mdot; multi-page Streamlit "
104         "application &mdot; CNN trained from scratch in NumPy</div>", unsafe_allow_html=True)
105
106
107 def confidence_style(c):
108     if c >= 0.80:
109         return GOOD, "High confidence"
110     if c >= 0.50:
111         return MID, "Moderate confidence"
112     return BAD, "Low confidence"
113
114

```

### 5.3 app.py — Model: NumPy CNN (inference only) (lines 115-183)

```

115 # =====
116 # 2. MODEL (inference-only CNN, NumPy)
117 # =====
118 def im2col(x, kh, kw, stride=1, pad=0):
119     N, H, W, C = x.shape
120     if pad > 0:
121         x = np.pad(x, ((0, 0), (pad, pad), (pad, pad), (0, 0)))
122         out_h = (H + 2 * pad - kh) // stride + 1
123         out_w = (W + 2 * pad - kw) // stride + 1
124         cols = np.zeros((N, out_h, out_w, kh, kw, C), dtype=x.dtype)
125         for i in range(kh):
126             for j in range(kw):
127                 cols[:, :, i, j, :] = x[:, i:i + stride * out_h:stride, j:j + stride * out_w:stride, :]
128     return cols.reshape(N, out_h, out_w, kh * kw * C), out_h, out_w
129
130
131 def conv_forward(x, W, b, stride=1, pad=1):
132     N = x.shape[0]
133     cols, oh, ow = im2col(x, W.shape[0], W.shape[0], stride, pad)
134     out = cols.reshape(N * oh * ow, -1) @ W.reshape(-1, W.shape[-1]) + b
135     return out.reshape(N, oh, ow, W.shape[-1])
136
137
138 def maxpool_forward(x, size=2, stride=2):
139     N, H, W, C = x.shape
140     oh, ow = H // stride, W // stride
141     x = x[:, :oh * stride, :ow * stride, :]
142     out = np.zeros((N, oh, ow, C), dtype=x.dtype)
143     for i in range(oh):
144         for j in range(ow):
145             out[:, i, j, :] = x[:, i * stride:i * stride + size, j * stride:j * stride + size, :].max(axis=(1, 2))
146     return out
147
148
149 def softmax(z):
150     e = np.exp(z - z.max(axis=1, keepdims=True))
151     return e / e.sum(axis=1, keepdims=True)
152
153
154 class DigitCNN:
155     """Conv -> ReLU -> Pool -> Conv -> ReLU -> Pool -> Dense -> ReLU -> Dense -> Softmax."""
156
157     def __init__(self, path):
158         with open(path, "rb") as f:
159             raw = pickle.load(f)
160             self.w = {k: v for k, v in raw.items() if k != "_meta"}
161             self.meta = raw.get("_meta")
162             if self.meta is None:
163                 h = hashlib.sha256()
164                 for k in sorted(self.w):
165                     h.update(self.w[k].tobytes())
166             self.meta = {"augmented_training": False, "weights_hash": h.hexdigest()[:10]}
167
168     def predict_proba(self, x):
169         w = self.w
170         x = np.maximum(0, conv_forward(x, w["conv1_W"], w["conv1_b"]))
171         x = maxpool_forward(x)
172         x = np.maximum(0, conv_forward(x, w["conv2_W"], w["conv2_b"]))
173         x = maxpool_forward(x)
174         x = np.maximum(0, x.reshape(x.shape[0], -1) @ w["fc1_W"] + w["fc1_b"])
175         return softmax(x @ w["fc2_W"] + w["fc2_b"])
176
177     def layer_table(self):
178         rows = []
179         for k, v in self.w.items():
180             rows.append({"Parameter": k, "Shape": " × ".join(map(str, v.shape)), "Count": int(v.size)})
181         return pd.DataFrame(rows)
182
183

```

## 5.4 app.py — Preprocessing and test-time augmentation (lines 184-260)

```
184 # =====
185 # 3. PREPROCESSING (identical for training and inference)
186 # =====
187 def pixels_to_display_image(p, size=192):
188     arr = np.clip(p, 0, 16) / 16.0 * 255
189     return Image.fromarray(arr.astype(np.uint8)).resize((size, size), Image.NEAREST)
190
191
192 def crop_to_ink(gray, pad_frac=0.07):
193     arr = gray.astype(np.float64)
194     if arr.max() <= arr.min():
195         return None
196     norm = (arr - arr.min()) / (arr.max() - arr.min()) * 255.0
197     mask = norm > max(25.0, norm.max() * 0.18)
198     if not mask.any():
199         return None
200     ys, xs = np.where(mask)
201     y0, y1, x0, x1 = ys.min(), ys.max(), xs.min(), xs.max()
202     py, px = max(1, int((y1 - y0 + 1) * pad_frac)), max(1, int((x1 - x0 + 1) * pad_frac))
203     y0, y1 = max(0, y0 - py), min(norm.shape[0] - 1, y1 + py)
204     x0, x1 = max(0, x0 - px), min(norm.shape[1] - 1, x1 + px)
205     return norm[y0:y1 + 1, x0:x1 + 1]
206
207
208 def shift_2d(arr, sy, sx):
209     out = np.zeros_like(arr)
210     h, w = arr.shape
211     out[max(0, sy):min(h, h + sy), max(0, sx):min(w, w + sx)] = \
212         arr[max(0, -sy):min(h, h - sy), max(0, -sx):min(w, w - sx)]
213     return out
214
215
216 def center_by_mass(c):
217     total = c.sum()
218     if total <= 0:
219         return c
220     yy, xx = np.mgrid[0:8, 0:8]
221     cy, cx = (yy * c).sum() / total, (xx * c).sum() / total
222     return shift_2d(c, int(round(3.5 - cy)), int(round(3.5 - cx)))
223
224
225 def dilate3(a):
226     return np.maximum.reduce([shift_2d(a, dy, dx) for dy in (-1, 0, 1) for dx in (-1, 0, 1)])
227
228
229 def erode3(a):
230     return np.minimum.reduce([shift_2d(a, dy, dx) for dy in (-1, 0, 1) for dx in (-1, 0, 1)])
231
232
233 def smart_preprocess_to_8x8(gray):
234     cropped = crop_to_ink(gray)
235     if cropped is None:
236         return np.zeros((8, 8))
237     ch, cw = cropped.shape
238     scale = 7.0 / max(ch, cw)
239     nh = int(np.clip(int(round(ch * scale)), 3.2, 8))
240     nw = int(np.clip(int(round(cw * scale)), 3.2, 8))
241     resized = np.array(Image.fromarray(cropped.astype(np.uint8)).resize((nw, nh), Image.LANCZOS)).astype(np.float64)
242     canvas = np.zeros((8, 8))
243     oy, ox = (8 - nh) // 2, (8 - nw) // 2
244     canvas[oy:oy + nh, ox:ox + nw] = resized
245     canvas = center_by_mass(canvas)
246     return canvas / canvas.max() * 16.0 if canvas.max() > 0 else canvas
247
248
249 def to_input(X):
250     return (np.clip(X, 0, 16) / 16.0).reshape(-1, 8, 8, 1).astype(np.float32)
251
252
253 def predict_with_tta(model, pixels):
254     """Average predictions over ±lpx shifts and thicker/thinner strokes."""
255     base = np.clip(pixels, 0, 16)
256     variants = [base] + [shift_2d(base, dy, dx) for dy in (-1, 0, 1) for dx in (-1, 0, 1) if (dy, dx) != (0, 0)]
257     variants += [np.clip(dilate3(base), 0, 16), np.clip(erode3(base), 0, 16)]
258     return model.predict_proba(to_input(np.stack(variants))).mean(axis=0)
259
260
```

## 5.5 app.py — Augmentation recipe (lines 261-309)

```
261 # =====
262 # 4. AUGMENTATION (same recipe used at training time)
263 # =====
264 def elastic_warp(arr, rng, upsample):
265     h, w = arr.shape
266     yy, xx = np.mgrid[0:h, 0:w].astype(np.float64)
267     amp = upsample * 0.035
268     fy, fx = rng.uniform(1.5, 3.0), rng.uniform(1.5, 3.0)
269     py, px = rng.uniform(0, 2 * np.pi), rng.uniform(0, 2 * np.pi)
270     dx = amp * np.sin(2 * np.pi * fy * yy / h + py)
271     dy = amp * np.sin(2 * np.pi * fx * xx / w + px)
272     return arr[np.clip(yy + dy, 0, h - 1).astype(np.int32), np.clip(xx + dx, 0, w - 1).astype(np.int32)]
273
274
275 def augment_one(base, rng, upsample=160):
276     img = Image.fromarray((np.clip(base, 0, 16) / 16.0 * 255.0).astype(np.uint8)).resize((upsample, upsample), Image.LANCZOS)
277     img = img.rotate(rng.uniform(-22, 22), resample=Image.BILINEAR, fillcolor=0)
278     if rng.random() < 0.3:
279         if rng.random() < 0.5:
280             sx, sy = rng.uniform(0.45, 0.7), rng.uniform(1.3, 1.7)
281         else:
282             sx, sy = rng.uniform(1.3, 1.7), rng.uniform(0.45, 0.7)
283     else:
284         sx, sy = rng.uniform(0.65, 1.4), rng.uniform(0.65, 1.4)
285     nw, nh = max(4, int(upsample * sx)), max(4, int(upsample * sy))
```

```

286 img = img.resize((nw, nh), Image.LANCZOS)
287 canvas = Image.new("L", (upsample, upsample), 0)
288 px = (upsample - nw) // 2 if nw <= upsample else 0
289 py = (upsample - nh) // 2 if nh <= upsample else 0
290 if nw > upsample or nh > upsample:
291     left, top = max(0, (nw - upsample) // 2), max(0, (nh - upsample) // 2)
292     img = img.crop((left, top, left + min(nw, upsample), top + min(nh, upsample)))
293     nw, nh = img.size
294     px, py = (upsample - nw) // 2, (upsample - nh) // 2
295 canvas.paste(img, (px, py))
296 arr = np.array(canvas).astype(np.float64)
297 arr = shift_2d(arr, int(rng.integers(-upsample // 7, upsample // 7 + 1)),
298               int(rng.integers(-upsample // 7, upsample // 7 + 1)))
299 if rng.random() < 0.6:
300     arr = elastic_warp(arr, rng, upsample)
301 if rng.random() < 0.55:
302     for _ in range(int(rng.integers(1, 3))):
303         arr = dilate3(arr)
304 elif rng.random() < 0.35:
305     arr = erode3(arr)
306 arr = np.clip(arr + rng.normal(0, 7, size=arr.shape), 0, 255)
307 return smart_preprocess_to_8x8(arr)
308
309

```

## 5.6 app.py — Cached loaders and live evaluation (lines 310–378)

```

310 # =====
311 # 5. CACHED LOADERS + LIVE EVALUATION
312 # =====
313 @st.cache_resource
314 def get_model():
315     return DigitCNN(MODEL_PATH)
316
317
318 @st.cache_data
319 def get_dataset():
320     d = load_digits()
321     return d.images, d.target
322
323
324 @st.cache_data(show_spinner="Evaluating the frozen model on clean + augmented validation data...")
325 def run_evaluation(augs_per_image=6, seed=7):
326     """Replaces the old evaluate.py + eval_results.pkl: computed once, cached."""
327     model = get_model()
328     images, labels = get_dataset()
329     rng = np.random.default_rng(seed)
330
331     idx = np.random.default_rng(42).permutation(len(images)) # same 85/15 split as training
332     val_idx = idx[int(len(images) * 0.85):]
333     vi, vl = images[val_idx], labels[val_idx]
334
335     X_clean = np.stack([smart_preprocess_to_8x8(np.clip(i, 0, 16) / 16.0 * 255.0) for i in vi])
336     p_clean = model.predict_proba(to_input(X_clean))
337     pred_clean = p_clean.argmax(1)
338
339     imgs, ys = [], []
340     for im, lb in zip(vi, vl):
341         for _ in range(augs_per_image):
342             imgs.append(augment_one(im, rng))
343             ys.append(lb)
344     X_aug, y_aug = np.stack(imgs), np.array(ys)
345     p_aug = model.predict_proba(to_input(X_aug))
346     pred_aug = p_aug.argmax(1)
347     conf_aug = p_aug[np.arange(len(p_aug)), pred_aug]
348     correct = pred_aug == y_aug
349
350     edges = np.linspace(0, 1, 11)
351     cal = []
352     for b in range(10):
353         m = (conf_aug >= edges[b]) & ((conf_aug < edges[b + 1]) if b < 9 else (conf_aug <= edges[b + 1]))
354         if m.sum():
355             cal.append({"conf": float(conf_aug[m].mean()), "acc": float(correct[m].mean()), "n": int(m.sum())})
356     ece = float(sum(c["n"] * abs(c["acc"] - c["conf"]) for c in cal) / len(y_aug))
357
358     wrong = np.where(~correct)[0]
359     rng.shuffle(wrong)
360     errors = [{"image": X_aug[i], "true": int(y_aug[i]), "pred": int(pred_aug[i]),
361               "conf": float(conf_aug[i])} for i in wrong[:12]]
362
363     mean_images = [np.stack([smart_preprocess_to_8x8(np.clip(im, 0, 16) / 16.0 * 255.0)
364                             for im in images[labels == d]].mean(0) for d in range(10)]
365
366     def pack(y, p):
367         return {"acc": float((y == p).mean()),
368               "cm": confusion_matrix(y, p, labels=list(range(10))),
369               "report": classification_report(y, p, labels=list(range(10)), output_dict=True, zero_division=0),
370               "n": int(len(y))}
371
372     return {"clean": pack(vl, pred_clean), "aug": pack(y_aug, pred_aug), "calibration": cal, "ece": ece,
373           "errors": errors, "mean_images": mean_images,
374           "class_counts": [int((labels == d).sum()) for d in range(10)],
375           "n_total": int(len(images)), "n_val": int(len(vl)), "augs_per_image": augs_per_image,
376           "conf_all": conf_aug, "correct_all": correct}
377
378

```

## 5.7 app.py — Sidebar badge and Home page (lines 379-426)

```
379 # =====
380 # 6. PAGES
381 # =====
382 def sidebar_badge(meta):
383     with st.sidebar:
384         st.markdown("### <U+1F522> Model status")
385         if meta.get("augmented_training"):
386             st.success(f"***Robust model loaded** \n\nFingerprint: `{meta.get('weights_hash', '?')}`\n\n"
387                       f"Clean {meta.get('clean_val_accuracy', 0)*100:.1f}% · "
388                       f"Messy {meta.get('augmented_val_accuracy', 0)*100:.1f}%")
389         else:
390             st.warning(f"Legacy model — fingerprint `{meta.get('weights_hash', '?')}`")
391
392
393 # ----- HOME -----
394 def page_home():
395     model, meta = get_model(), get_model().meta
396     hero("Advanced Multi-Page Streamlit Application", "Digit Intelligence Suite",
397         "A deep-learning application with a live classifier, a model-performance dashboard, "
398         "a dataset explorer and built-in documentation — one model, one design system.")
399     metric_row([
400         ("Clean-digit accuracy", f"{meta.get('clean_val_accuracy', 0)*100:.2f}%", "held-out validation split"),
401         ("Messy accuracy", f"{meta.get('augmented_val_accuracy', 0)*100:.2f}%", "rotated / scaled / noisy"),
402         ("Training epochs", f"{meta.get('epochs', '?')}", f"{meta.get('augments_per_image', '?')}\n augmentation"),
403         ("Parameters", f"{int(model.layer_table()['Count'].sum()):,}", "trainable weights"),
404     ])
405
406     st.markdown("### What's inside")
407     c1, c2 = st.columns(2)
408     cards = [
409         (c1, "<U+1F9EA> Try It Live", "Classify a sample, draw a digit, or upload a photo — with test-time augmentation and a session history.", "page_live"),
410         (c1, "<U+1F4CA> Model Dashboard", "Confusion matrices, precision/recall/F1, calibration, top confusions and misclassified examples.", "pages."),
411         (c2, "<U+1F50D> Dataset Explorer", "Class balance, per-class mean digits, raw browser and a live augmentation preview.", "page_data"),
412         (c2, "<U+1F4C4> Documentation", "Objective, approach, deliverables, architecture and known limitations.", "page_docs"),
413     ]
414     for col, title, text, key in cards:
415         with col, st.container(border=True):
416             st.markdown(f"### {title}")
417             st.write(text)
418             st.page_link(PAGES[key], label=f"Open {title.split(' ', 1)[1]}")
419
420     st.markdown("### Model architecture")
421     st.code("Input 8x8x1 → Conv(3x3, same) → ReLU → MaxPool 2x2 → Conv(3x3, same) → ReLU → "
422           "MaxPool 2x2 → Flatten → Dense → ReLU → Dense(10) → Softmax", language=None, wrap_lines=True)
423     st.dataframe(model.layer_table(), hide_index=True, width="stretch")
424     footer()
```

## 5.8 app.py — Try It Live page (lines 427-630)

```
427 # ----- TRY IT LIVE -----
428 def render_prediction(pixels, source):
429     model = get_model()
430     with st.spinner("Running the CNN with test-time augmentation..."):
431         probs = predict_with_tta(model, pixels)
432         pred, conf = int(np.argmax(probs)), float(probs.max())
433         color, word = confidence_style(conf)
434
435         key = hashlib.md5(np.round(pixels, 2).tobytes()).hexdigest()
436         hist = st.session_state.setdefault("history", [])
437         last_logged = st.session_state.setdefault("last_logged", {})
438         # Log once per distinct input per source. Comparing only with hist[-1] re-logged a
439         # still-displayed sample result every time another tab added a newer entry.
440         if last_logged.get(source) != key:
441             hist.append({"key": key, "Source": source, "Prediction": pred, "Confidence": round(conf * 100, 1)})
442             last_logged[source] = key
443
444         st.markdown("### Result")
445         c1, c2, c3 = st.columns([1, 1.1, 1.3])
446         with c1:
447             st.image(pixels_to_display_image(pixels, 220), caption="What the model saw (8x8)", width="stretch")
448         with c2:
449             st.markdown(
450                 f'<div class="result-card" style="--card-border:{color}55; --card-bg:{color}14;">'
451                 f'<div class="result-label">Predicted digit</div>'
452                 f'<div class="result-digit" style="color:{color}">{pred}</div>'
453                 f'<span class="confidence-pill" style="background:{color}22; color:{color}">'
454                 f'{conf*100:.1f}% · {word}</span></div>', unsafe_allow_html=True)
455             if conf < 0.5:
456                 st.caption("⚠ Not very sure — try a bolder, more centered digit.")
457         with c3:
458             st.markdown("### Top 3 guesses")
459             for rank, d in enumerate(np.argsort(probs)[::-1][1:3]):
460                 st.markdown(f'<U+1F947><U+1F948><U+1F949>[rank] **{d}** — {probs[d]*100:.1f}%')
461                 st.progress(float(np.clip(probs[d], 0, 1)))
462
463         with st.expander("Full probability breakdown"):
464             df = pd.DataFrame({"Digit": DIGITS, "Probability": probs, "Top": [d == pred for d in range(10)]})
465             st.altair_chart(
466                 alt.Chart(df).mark_bar(cornerRadiusTopLeft=4, cornerRadiusTopRight=4).encode(
467                     x=alt.X("Digit:N", sort=None),
468                     y=alt.Y("Probability:Q", scale=alt.Scale(domain=[0, 1])),
469                     color=alt.condition(alt.datum.Top, alt.value(ACCENT), alt.value("#5B7FDE88")),
470                     tooltip=["Digit", alt.Tooltip("Probability:Q", format=".1%")]).properties(height=240,
471                     width="stretch")
472
473
474 def page_live():
475     sample_images, sample_labels = get_dataset()
476     hero("Interactive", "Try It Live",
477         "Classify a digit three ways: pick a labeled sample, draw one by hand, or upload a photo. "
```

```

478     "Every mode runs the same crop → scale → center pipeline plus test-time augmentation.")
479
480     t_samples, t_draw, t_upload = st.tabs(["<U+1F5BC><U+FE0F> Sample Digits", "<U+FE0F> Draw Your Own", "<U+1F4E4> Upload a Photo"])
481
482     # ---- samples
483     with t_samples:
484         steps("Filter by digit (optional), then click <b>Use this</b> under a thumbnail.",
485              "Press <b>Predict</b> to see what the model thinks.")
486         f_col, s_col = st.columns([2, 3])
487         with f_col:
488             dfilter = st.selectbox("Filter gallery by true digit", ["All digits"] + DIGITS)
489             cand = np.arange(len(sample_images)) if dfilter == "All digits" else np.where(sample_labels == int(dfilter))[0]
490             st.session_state.setdefault("gallery_seed", 0)
491             if st.session_state.get("selected_sample") not in cand:
492                 st.session_state.selected_sample = int(cand[0])
493             st.session_state.show_sample_result = False
494         with s_col:
495             st.write("")
496             if st.button("<U+1F500> Shuffle gallery"):
497                 st.session_state.gallery_seed += 1
498             show = np.random.RandomState(st.session_state.gallery_seed).choice(cand, min(8, len(cand)), replace=False)
499             cols = st.columns(4)
500             for i, idx in enumerate(show):
501                 with cols[i % 4], st.container(border=True):
502                     st.image(pixels_to_display_image(sample_images[idx], 150), width="stretch")
503                     st.caption(f"True label: **{sample_labels[idx]}**")
504                     if st.button("Use this", key=f"use_{idx}", width="stretch"):
505                         st.session_state.selected_sample = int(idx)
506                         st.session_state.show_sample_result = False
507             st.divider()
508             sel = st.session_state.selected_sample
509             st.markdown(f"***Selected** - index `{sel}`, true label **{sample_labels[sel]}**")
510             l, r = st.columns([1, 3])
511             l.image(pixels_to_display_image(sample_images[sel], 200), width="stretch")
512             with r:
513                 st.write("")
514                 if st.button("<U+1F680> Predict", key="p_sample", type="primary", width="stretch"):
515                     st.session_state.show_sample_result = True
516             if st.session_state.get("show_sample_result"):
517                 st.divider()
518                 render_prediction(sample_images[sel].copy(), "Sample")
519
520     # ---- draw
521     with t_draw:
522         if CANVAS_AVAILABLE:
523             steps("Draw a single digit (0-9) with mouse or touch.", "Press <b>Predict</b> when happy with it.")
524             d_col, p_col = st.columns([3, 2])
525             canvas_result, err = None, None
526             with d_col:
527                 brush = st.slider("Brush size", 8, 40, 22)
528                 try:
529                     canvas_result = st_canvas(fill_color="white", stroke_width=brush, stroke_color="white",
530                                              background_color="black", height=380, width=380,
531                                              drawing_mode="freedraw", return_image_data=True, key="canvas")
532                 except Exception as e:
533                     err = str(e)
534             go = st.button("<U+1F680> Predict", key="p_canvas", type="primary") if canvas_result is not None else False
535             if err:
536                 st.warning(f"⚠ The drawing canvas couldn't load (version mismatch with Streamlit). Detail: `{err}`")
537             else:
538                 drawn = None
539                 data = canvas_result.image_data
540                 if data is not None and data[:, :, :3].sum() > 0:
541                     gray = np.array(Image.fromarray(data.astype(np.uint8)).convert("L"))
542                     drawn = smart_preprocess_to_8x8(gray)
543                 with p_col:
544                     st.markdown("#### Live model-input preview")
545                     if drawn is not None and drawn.sum() > 0:
546                         st.image(pixels_to_display_image(drawn, 300), width="stretch")
547                     else:
548                         st.info("Start drawing to see a live preview.")
549                 if go:
550                     if drawn is None or drawn.sum() == 0:
551                         st.warning("The canvas looks empty – draw a digit first!")
552                     else:
553                         st.divider()
554                         render_prediction(drawn, "Drawn")
555             else:
556                 st.info("Drawing canvas not installed ('pip install streamlit-drawable-canvas'). "
557                      "Use this pixel-brush grid instead:")
558                 st.session_state.setdefault("grid", np.zeros((8, 8), dtype=np.int32))
559                 strength = st.slider("Brush strength", 2, 16, 8, key="fb_brush")
560                 g_l, g_r = st.columns([3, 1])
561                 with g_r:
562                     if st.button("Load example '0'", width="stretch"):
563                         st.session_state.grid = sample_images[np.where(sample_labels == 0)[0][0]].astype(np.int32).copy()
564                     if st.button("Clear grid", width="stretch"):
565                         st.session_state.grid = np.zeros((8, 8), dtype=np.int32)
566                     st.image(pixels_to_display_image(st.session_state.grid), width="stretch")
567                 with g_l:
568                     for r_ in range(8):
569                         rc = st.columns(8)
570                         for c_ in range(8):
571                             v = int(st.session_state.grid[r_, c_])
572                             if rc[c_].button("<U+2B1B>" if v == 0 else ("<U+2B1C>" if v >= 12 else "■"), key=f"cell_{r_}_{c_}"):
573                                 st.session_state.grid[r_, c_] = min(16, v + strength)
574                             st.rerun()
575                 if st.button("<U+1F680> Predict", key="p_grid", type="primary"):
576                     if st.session_state.grid.sum() == 0:
577                         st.warning("The grid is empty – click some cells first!")
578                     else:
579                         st.divider()
580                         render_prediction(st.session_state.grid.astype(np.float64), "Grid")
581
582     # ---- upload
583     with t_upload:
584         steps("Upload a clear photo or scan with a single handwritten digit.",
585              "Toggle <b>Invert colors</b> if the preview looks wrong.")
586         u_col, o_col = st.columns([2, 1])
587         invert = o_col.checkbox("Invert colors", value=True, help="ON for a dark digit on a light background.")

```

```

588 up = u_col.file_uploader("Choose an image", type=["png", "jpg", "jpeg"])
589 if up is not None:
590     gray = np.array(Image.open(up).convert("L")).astype(np.float64)
591     if invert:
592         gray = 255.0 - gray
593     crop, arr = crop_to_ink(gray), smart_preprocess_to_8x8(gray)
594     st.divider()
595     a, b, c = st.columns(3)
596     a.markdown("***Original upload***")
597     a.image(up, width="stretch")
598     b.markdown("***Auto-cropped***")
599     if crop is not None:
600         b.image(Image.fromarray(crop.astype(np.uint8)).resize((240, 240), Image.NEAREST), width="stretch")
601     else:
602         b.warning("No clear digit found – try the invert toggle.")
603     c.markdown("***Model input***")
604     c.image(pixels_to_display_image(arr, 240), width="stretch")
605     if st.button("<U+1F680> Predict", key="p_upload", type="primary"):
606         if arr.sum() == 0:
607             st.warning("No digit detected – try another photo.")
608         else:
609             st.divider()
610             render_prediction(arr, "Upload")
611     else:
612         st.info("<U+1F4E4> Upload a PNG or JPG of a handwritten digit to get started.")
613
614 hist = st.session_state.get("history", [])
615 if hist:
616     st.markdown("### Session history")
617     h_df = pd.DataFrame(hist).drop(columns="key")
618     h_df.index = h_df.index + 1
619     hc1, hc2 = st.columns([4, 1])
620     hc1.dataframe(h_df.iloc[:-1], width="stretch")
621     with hc2:
622         st.download_button("⬇ Download CSV", h_df.to_csv(index_label="#"), "prediction_history.csv", "text/csv",
623                             width="stretch")
624     if st.button("Clear history", width="stretch"):
625         st.session_state.history = []
626         st.session_state.last_logged = {}
627         st.session_state.show_sample_result = False # otherwise the sample would be re-logged
628         st.rerun()
629
630

```

## 5.9 app.py — Model Dashboard page (lines 631-728)

```

631 # ----- MODEL DASHBOARD -----
632 def confusion_chart(cm, title):
633     df = pd.DataFrame(cm, index=DIGITS, columns=DIGITS).reset_index().melt(
634         id_vars="index", var_name="Predicted", value_name="Count").rename(columns={"index": "True"})
635     enc = dict(x=alt.X("Predicted:N", sort=DIGITS), y=alt.Y("True:N", sort=DIGITS))
636     rect = alt.Chart(df).mark_rect().encode(
637         *enc, color=alt.Color("Count:Q", scale=alt.Scale(scheme="blues")), tooltip=["True", "Predicted", "Count"])
638     text = alt.Chart(df).mark_text(baseline="middle").encode(
639         *enc, text="Count:Q",
640         color=alt.condition(alt.datum.Count > cm.max() / 2, alt.value("white"), alt.value("black")))
641     return (rect + text).properties(height=380, title=title)
642
643
644 def per_class_chart(report, title):
645     rows = [{"Digit": d, "Precision": report[d]["precision"], "Recall": report[d]["recall"], "F1": report[d]["f1-score"]}
646             for d in DIGITS]
647     df = pd.DataFrame(rows).melt(id_vars="Digit", var_name="Metric", value_name="Score")
648     return alt.Chart(df).mark_bar().encode(
649         x=alt.X("Digit:N", sort=DIGITS), y=alt.Y("Score:Q", scale=alt.Scale(domain=[0, 1])),
650         color=alt.Color("Metric:N", scale=alt.Scale(range=[ACCENT, NAVY, GOOD])), xOffset="Metric:N",
651         tooltip=["Digit", "Metric", alt.Tooltip("Score:Q", format=".3f")]).properties(height=320, title=title)
652
653
654 def top_confusions(cm, k=5):
655     pairs = [(i, j, int(cm[i, j])) for i in range(10) for j in range(10) if i != j and cm[i, j] > 0]
656     pairs.sort(key=lambda t: -t[2])
657     return pd.DataFrame([{"True": a, "Predicted as": b, "Count": c} for a, b, c in pairs[:k]])
658
659
660 def page_dash():
661     meta = get_model().meta
662     res = run_evaluation()
663     hero("Dashboard", "Model Performance Dashboard",
664         "Confusion matrices, per-class precision / recall / F1, confidence calibration and worked "
665         "misclassification examples – computed live from the frozen model.")
666     metric_row([
667         ("Clean accuracy", f"{res['clean']['acc']*100:.2f}%", f"{res['n_val']} held-out images"),
668         ("Messy accuracy", f"{res['aug']['acc']*100:.2f}%", f"{res['aug']['n']} augmented variants"),
669         ("Macro F1 (messy)", f"{res['aug']['report']['macro avg']['f1-score']:.3f}", "average over 10 classes"),
670         ("Calibration error", f"{res['ece']*100:.2f}%", "ECE on messy set (lower = better)"),
671     ])
672     st.caption(f"Training-time numbers stored in the model file: clean "
673              f"{meta.get('clean_val_accuracy', 0)*100:.2f}% / messy {meta.get('augmented_val_accuracy', 0)*100:.2f}%. "
674              "Close agreement with the recomputed numbers above indicates the model generalizes.")
675
676     tab_c, tab_m, tab_t = st.tabs(["Clean validation set", "Messy / augmented set", "Per-class table"])
677     for tab, key, name in ((tab_c, "clean", "clean"), (tab_m, "aug", "messy")):
678         with tab:
679             a, b = st.columns(2)
680             a.altair_chart(confusion_chart(res[key]["cm"], f"Confusion matrix – {name}"), width="stretch")
681             b.altair_chart(per_class_chart(res[key]["report"], f"Precision / Recall / F1 – {name}"), width="stretch")
682             st.markdown("***Most frequent confusions***")
683             tc = top_confusions(res[key]["cm"])
684             if tc.empty:
685                 st.success("No misclassifications in this set.")
686             else:
687                 st.dataframe(tc, hide_index=True, width="stretch")
688     with tab_t:
689         rows = []
690         for d in DIGITS:
691             rc, ra = res["clean"]["report"][d], res["aug"]["report"][d]

```

```

692         rows.append({"Digit": d, "Clean P": rc["precision"], "Clean R": rc["recall"], "Clean F1": rc["f1-score"],
693                     "Messy P": ra["precision"], "Messy R": ra["recall"], "Messy F1": ra["f1-score"],
694                     "Messy support": int(ra["support"])})
695     tbl = pd.DataFrame(rows)
696     st.dataframe(tbl.style.format({c: "{:.3f}" for c in tbl.columns if c not in ("Digit", "Messy support")}),
697                 hide_index=True, width="stretch")
698     st.download_button("⬇ Download per-class report (CSV)", tbl.to_csv(index=False), "per_class_report.csv", "text/csv")
699
700     st.markdown("### Confidence calibration")
701     st.caption("Bars should follow the dashed diagonal: when the model says 70% sure, it should be right ~70% of the time.")
702     cal = pd.DataFrame(res["calibration"]).rename(columns={"conf": "Mean confidence", "acc": "Actual accuracy", "n": "Count"})
703     bars = alt.Chart(cal).mark_bar(color=ACCENT, opacity=0.85, size=22).encode(
704         x=alt.X("Mean confidence:Q", scale=alt.Scale(domain=[0, 1], nice=False), title="Mean confidence"),
705         y=alt.Y("Actual accuracy:Q", scale=alt.Scale(domain=[0, 1], nice=False), title="Actual accuracy"),
706         tooltip=[alt.Tooltip("Mean confidence:Q", format=".2f"), alt.Tooltip("Actual accuracy:Q", format=".2f"), "Count"])
707     diag = alt.Chart(pd.DataFrame({"x": [0, 1], "y": [0, 1]})).mark_line(strokeDash=[4, 4], color="#999").encode(
708         x=alt.X("x:Q", scale=alt.Scale(domain=[0, 1], nice=False), title="Mean confidence"),
709         y=alt.Y("y:Q", scale=alt.Scale(domain=[0, 1], nice=False), title="Actual accuracy"))
710     cc1, cc2 = st.columns(2)
711     cc1.altair_chart((bars + diag).properties(height=300, title="Reliability diagram"), width="stretch")
712     hdf = pd.DataFrame({"Confidence": res["conf_all"], "Outcome": np.where(res["correct_all"], "Correct", "Wrong")})
713     cc2.altair_chart(
714         alt.Chart(hdf).mark_bar(opacity=0.85).encode(
715             x=alt.X("Confidence:Q", bin=alt.Bin(maxbins=20)), y=alt.Y("count():Q", title="Predictions"),
716             color=alt.Color("Outcome:N", scale=alt.Scale(domain=["Correct", "Wrong"], range=[GOOD, BAD]))
717         ).properties(height=300, title="Confidence distribution: correct vs wrong"), width="stretch")
718
719     st.markdown("### Misclassification examples")
720     st.caption("Genuine errors from the messy evaluation set – not cherry-picked successes.")
721     cols = st.columns(4)
722     for i, ex in enumerate(res["errors"][:8]):
723         with cols[i % 4]:
724             st.image(pixels_to_display_image(ex["image"], 130), width="stretch")
725             st.caption(f"True **{ex['true']}** → predicted **{ex['pred']}** ({{ex['conf']*100:.0f}}%)")
726 footer()
727
728

```

## 5.10 app.py — Dataset Explorer page (lines 729-786)

```

729 # ----- DATASET EXPLORER -----
730 def page_data():
731     images, labels = get_dataset()
732     res = run_evaluation()
733     hero("Exploration", "Dataset Explorer",
734         "The UCI handwritten-digits dataset the model was trained on, and the augmentation pipeline "
735         "that turns 1,797 clean 8x8 digits into something closer to real freehand input.")
736     metric_row(["Images", f"{res['n_total']:,}", "8x8 grayscale"), ("Classes", "10", "digits 0-9"),
737               ("Per class", f"{min(res['class_counts'])}-{max(res['class_counts'])}", "reasonably balanced"),
738               ("Pixel range", "0-16", "4-bit intensity"))
739
740     st.markdown("### Class balance")
741     dist = pd.DataFrame({"Digit": DIGITS, "Count": res["class_counts"]})
742     st.altair_chart(alt.Chart(dist).mark_bar(color=NAVY, cornerRadiusTopLeft=4, cornerRadiusTopRight=4).encode(
743         x=alt.X("Digit:N", sort=None), y="Count:Q", tooltip=["Digit", "Count"]).properties(height=260), width="stretch")
744
745     st.markdown("### What a “typical” digit looks like")
746     st.caption("Per-class mean image after the same crop/scale/center preprocessing the model sees at inference.")
747     cols = st.columns(5)
748     for d in range(10):
749         with cols[d % 5]:
750             st.image(pixels_to_display_image(res["mean_images"][d], 110), width="stretch")
751             st.caption(f"Digit {d}")
752
753     st.markdown("### Browse the raw dataset")
754     f = st.selectbox("Filter by true digit", ["All digits"] + DIGITS, key="explorer_filter")
755     idxs = np.arange(len(images)) if f == "All digits" else np.where(labels == int(f))[0]
756     if st.button("<U+1F500> Shuffle"):
757         st.session_state.explorer_seed = st.session_state.get("explorer_seed", 0) + 1
758     show = np.random.RandomState(st.session_state.get("explorer_seed", 0)).choice(idxs, min(10, len(idxs)), replace=False)
759     cols = st.columns(5)
760     for i, idx in enumerate(show):
761         with cols[i % 5]:
762             st.image(pixels_to_display_image(images[idx], 110), width="stretch")
763             st.caption(f"index {idx} · label {labels[idx]}")
764
765     st.markdown("### Augmentation pipeline preview")
766     st.caption("Rotation, independent-axis scaling, translation, elastic warp, stroke thickening/thinning and noise – "
767             "the same `augment_one` used at training time.")
768     a1, a2 = st.columns([1, 3])
769     with a1:
770         sidx = st.number_input("Sample index", 0, len(images) - 1, int(np.where(labels == 8)[0][0]), 1)
771         nv = st.slider("Number of variants", 3, 8, 6)
772         regen = st.button("<U+1F3B2> Generate variants", type="primary")
773     with a2:
774         st.markdown("***Original**")
775         st.image(pixels_to_display_image(images[sidx], 110))
776         if regen or "aug_cache" not in st.session_state or st.session_state.get("aug_idx") != sidx:
777             rng = np.random.default_rng()
778             st.session_state.aug_cache = [augment_one(images[sidx], rng) for _ in range(8)]
779             st.session_state.aug_idx = sidx
780         st.markdown("***Augmented variants**")
781         vc = st.columns(nv)
782         for i, v in enumerate(st.session_state.aug_cache[nv]):
783             vc[i].image(pixels_to_display_image(v, 100), width="stretch")
784 footer()
785
786

```

## 5.11 app.py — Documentation page (lines 787-835)

```
787 # ----- DOCUMENTATION -----
788 def page_docs():
789     hero("Reference", "Documentation", "Objective, approach, deliverables, architecture and known limitations.")
790     st.markdown("### Objective")
791     st.write("To develop a professional deep-learning application with multiple pages and interactive visualizations.")
792     st.markdown("### Reference dataset")
793     st.write("UCI Optical Recognition of Handwritten Digits (scikit-learn `load_digits`) – an image classification "
794             "dataset of 1,797 8x8 grayscale digits across 10 classes.")
795
796     st.markdown("### Approach → where it is implemented")
797     st.markdown("""
798 | Requirement | Implementation |
799 |---|---|
800 | Create a multi-page architecture | `st.navigation` with five pages (Home, Try It Live, Model Dashboard, Dataset Explorer, Document
801 | ation) in this one file |
802 | Add dashboards and visualizations | Altair confusion matrices, grouped P/R/F1 bars, reliability diagram, confidence histogram, cla
803 | ss balance, mean digits |
804 | Display model performance metrics | Accuracy, macro-F1, expected calibration error, per-class table, top confusions |
805 | Integrate charts and reports | Interactive tooltips, plus CSV downloads of the per-class report and prediction history |
806 | Improve navigation and user experience | Sidebar navigation, page-link cards, model-status badge, tabs, session history, spinners,
807 | empty-state guidance |
808 """)
809     st.markdown("### Deliverables")
810     st.markdown("- **Multi-page Streamlit application** – `app.py`\n"
811               "- **Dashboard visualizations** – *Model Dashboard* and *Dataset Explorer* pages\n"
812               "- **Project documentation** – this page")
813
814     st.markdown("### Methodology")
815     st.write("A small CNN (Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Dense → ReLU → Dense → Softmax) trained "
816            "from scratch in NumPy with hand-derived backpropagation. The identical crop-to-ink, independent-axis "
817            "scale and center-by-mass pipeline is used for training augmentation and for every live input, so "
818            "training and inference never drift apart. Live predictions average 11 test-time-augmentation variants "
819            "(±1px shifts, thicker/thinner strokes).")
820     st.markdown("### Project files")
821     st.markdown("""
822 | File | Role |
823 |---|---|
824 | `app.py` | The entire application (this file) |
825 | `cnn_digits_model.pkl` | Trained weights + metadata |
826 | `requirements.txt` | Python dependencies |
827 """)
828     st.markdown("### Known limitations")
829     st.markdown("""
830 - **Small, low-resolution data** – ~1,800 images at 8x8 is not production-grade.
831 - **Extreme proportions** – very tall/wide digits become ambiguous when compressed to 8x8.
832 - **Stylistic mismatch** – ornate or heavily cursive digits differ from the plain training style.
833 - **Evaluation recency** – metrics are computed from the model file currently loaded and cached; restart the app
834 after replacing `cnn_digits_model.pkl`.
835 """)
836     footer()
```

## 5.12 app.py — Entry point and navigation (lines 836-862)

```
836 # =====
837 # 7. ENTRY POINT
838 # =====
839 PAGES = {}
840
841
842 def main():
843     st.set_page_config(page_title="Digit Intelligence Suite", page_icon="<U+1F52D>", layout="wide",
844                       initial_sidebar_state="expanded")
845     st.markdown(CSS, unsafe_allow_html=True)
846     if not MODEL_PATH.exists():
847         st.error(f"Model file not found: `{MODEL_PATH.name}`. Put it in the same folder as app.py.")
848     st.stop()
849
850     PAGES["page_home"] = st.Page(page_home, title="Home", icon="<U+1F3E0>", url_path="home", default=True)
851     PAGES["page_live"] = st.Page(page_live, title="Try It Live", icon="<U+1F9EA>", url_path="live")
852     PAGES["page_dash"] = st.Page(page_dash, title="Model Dashboard", icon="<U+1F4CA>", url_path="dashboard")
853     PAGES["page_data"] = st.Page(page_data, title="Dataset Explorer", icon="<U+1F50D>", url_path="dataset")
854     PAGES["page_docs"] = st.Page(page_docs, title="Documentation", icon="<U+1F4C4>", url_path="docs")
855
856     nav = st.navigation(list(PAGES.values()))
857     sidebar_badge(get_model().meta)
858     nav.run()
859
860
861 if __name__ == "__main__":
862     main()
```

## 5.13 requirements.txt

```
streamlit>=1.53
numpy
pandas
altair
pillow
scikit-learn
streamlit-drawable-canvas
```

Note: `streamlit>=1.53` is the real minimum. `width="stretch"` fails on Streamlit 1.46 and 1.50 and works from 1.52, and the current `streamlit-drawable-canvas` release requires Streamlit 1.53 or newer.

## 6. Screenshots of the Interface

All screenshots below were captured from the running application (Streamlit 1.65.0, streamlit-drawable-canvas 0.13.0) using a headless Chromium browser driven by Playwright, with real mouse drawing, a real file chooser and real button clicks. The predictions, confidences and charts shown are genuine model output, not mock-ups. Two notes for transparency: the 'photo' uploaded in Section 6.5 is a synthetic stand-in generated from held-out dataset digit #566 (a '2'; upscaled, blurred, rotated and noised), and Streamlit's top toolbar is visible in Figure 1 and cropped out of the other figures. Each screenshot has its own page.

### 6.1 Home — Overview

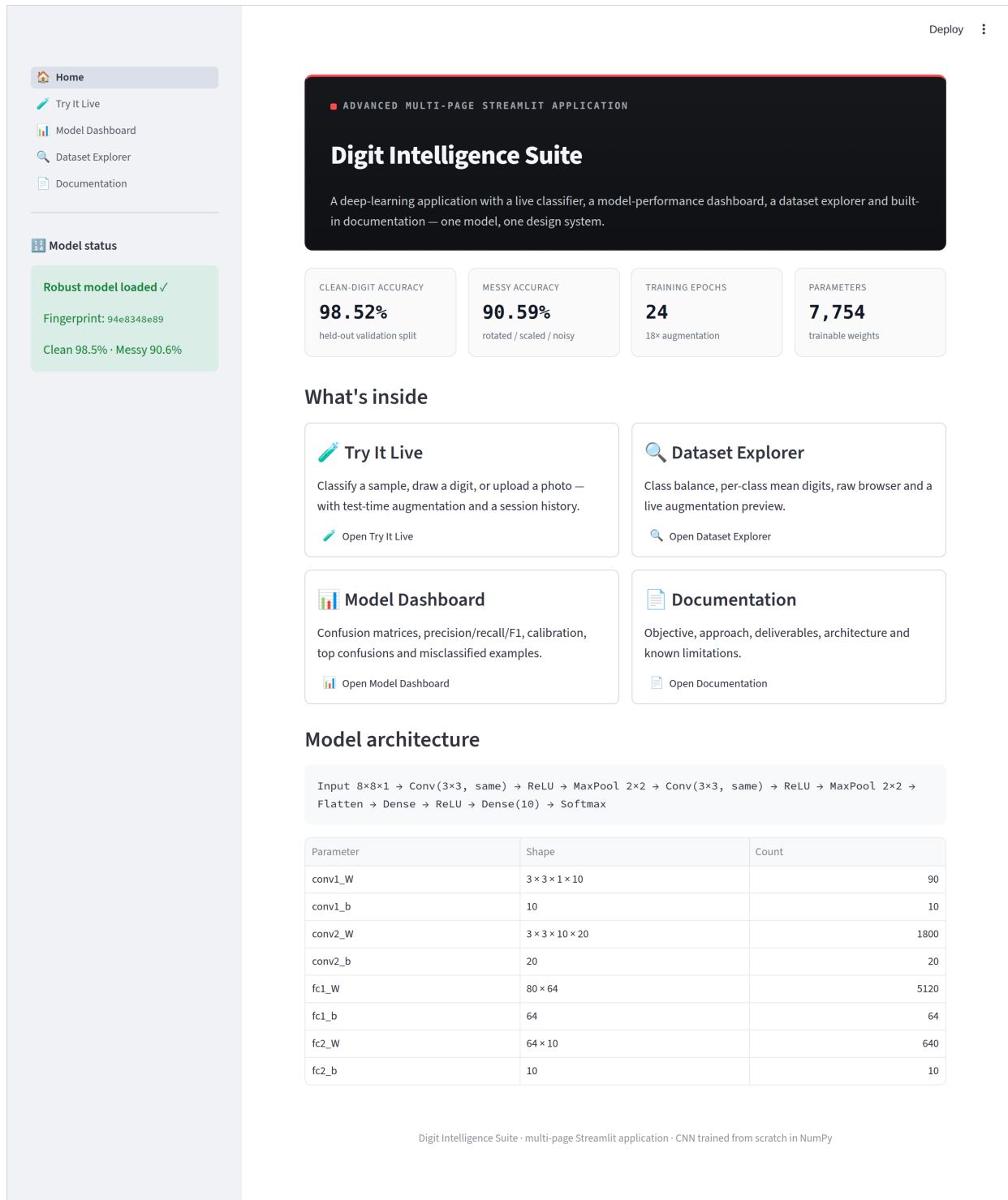


Figure 1: The full Home page: sidebar navigation with the model-status badge (fingerprint 94e8348e89), the hero banner, four headline cards (clean 98.52% and messy 90.59% are the accuracies recorded in the model file; 24 epochs with 18× augmentation; 7,754 parameters), the navigation cards, the wrapped architecture line and the parameter table.

6.2 Try It Live — Sample Gallery

INTERACTIVE

Try It Live

Classify a digit three ways: pick a labeled sample, draw one by hand, or upload a photo. Every mode runs the same crop → scale → center pipeline plus test-time augmentation.

Sample Digits

Draw Your Own

Upload a Photo

1

Filter by digit (optional), then click **Use this** under a thumbnail.

2

Press **Predict** to see what the model thinks.

Filter gallery by true digit

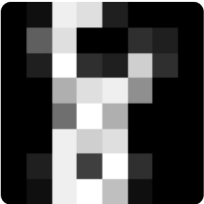
All digits

Shuffle gallery



True label: 2

Use this



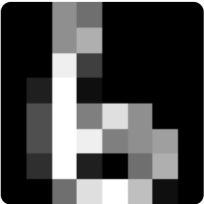
True label: 8

Use this



True label: 2

Use this



True label: 6

Use this



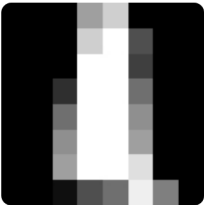
True label: 6

Use this



True label: 7

Use this



True label: 1

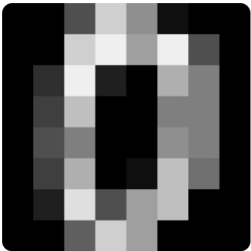
Use this



True label: 9

Use this

Selected — index 0, true label 0



Predict

Figure 2: Sample Digits tab: numbered step guidance, a filter by true digit, a shuffle button, eight labelled thumbnails with 'Use this' buttons, and the selected sample (index 0, true label 0) with its Predict button.

17 / 31

6.3 Try It Live — Sample Selection and Prediction Result

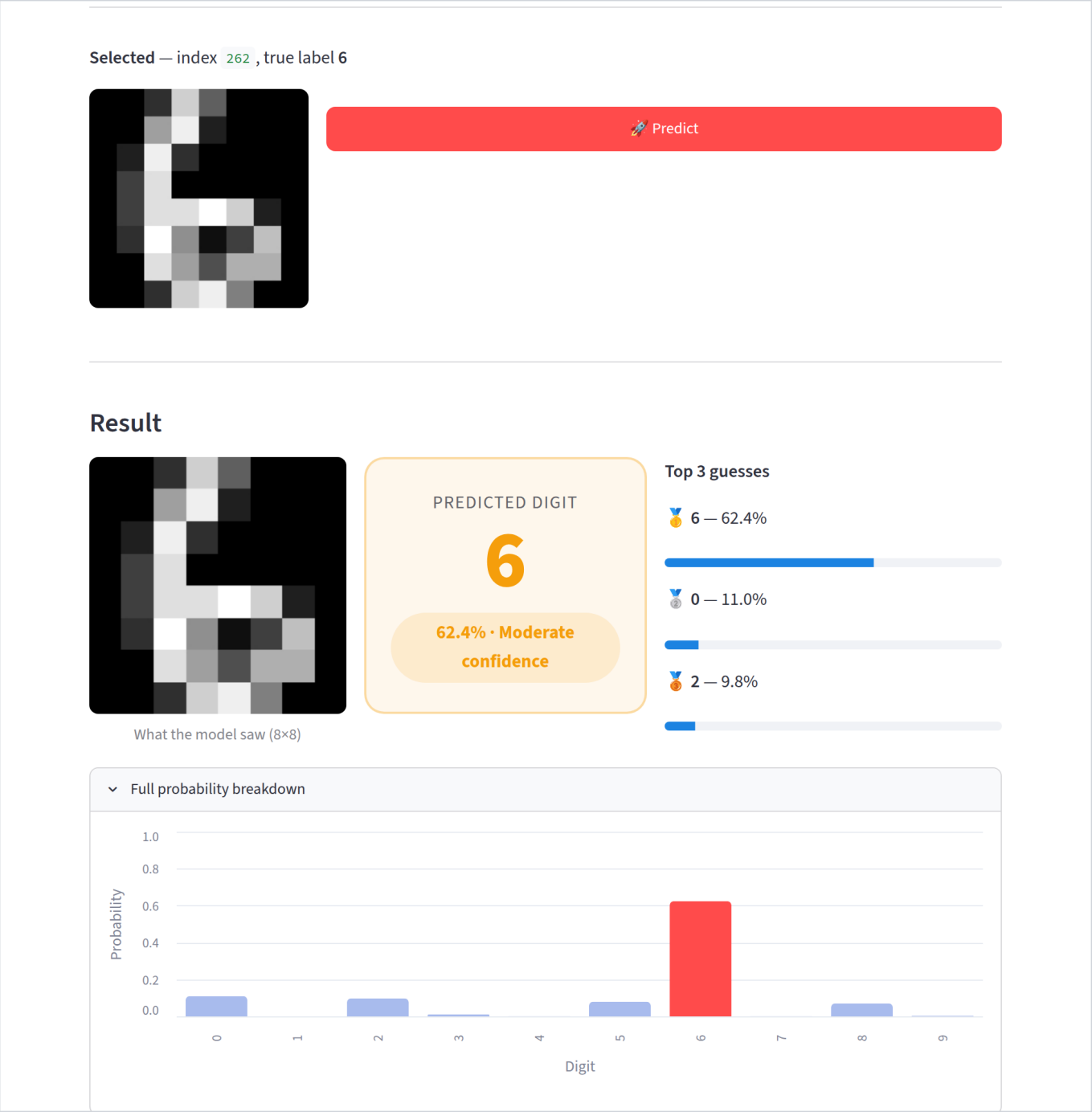


Figure 3: The selected sample (index 262, true label 6) with its Predict button, and the result: predicted 6 with 62.4 % confidence ('moderate'), the 8×8 image the model saw, the top-3 ranking (6, 0, 2) and the expanded full probability chart.

6.4 Try It Live — Drawing Canvas and Prediction

Sample Digits

Draw Your Own

Upload a Photo

1 Draw a single digit (0–9) with mouse or touch.

2 Press Predict when happy with it.

Brush size

22

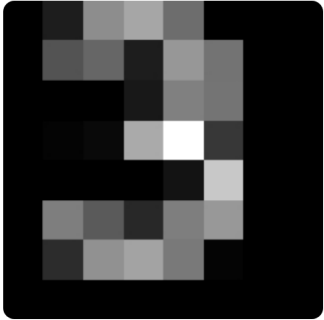


Predict

Live model-input preview



Result



What the model saw (8×8)

PREDICTED DIGIT

3

70.3% · Moderate confidence

Top 3 guesses

3 — 70.3%

0 — 13.0%

8 — 9.4%

> Full probability breakdown

Figure 4: A '3' drawn with the mouse next to the live 8×8 model-input preview, followed by the result: 3 with 70.3 % confidence (runners-up 0 at 13.0 % and 8 at 9.4 %).

6.5 Try It Live — Photo Upload Result and Session History

Sample Digits

Draw Your Own

Upload a Photo

1

Upload a clear photo or scan with a single handwritten digit.

2

Toggle **Invert colors** if the preview looks wrong.

Choose an image

sample\_photo.png

50.2KB

+

☒ Invert colors

Original upload

Auto-cropped

Model input

Predict

Result

What the model saw (8×8)

PREDICTED DIGIT

2

66.8% · Moderate confidence

Top 3 guesses

2 — 66.8%

9 — 10.7%

8 — 9.3%

> Full probability breakdown

Session history

|   | Source | Prediction | Confidence |
|---|--------|------------|------------|
| 3 | Upload | 2          | 66.8       |
| 2 | Drawn  | 3          | 70.3       |
| 1 | Sample | 6          | 62.4       |

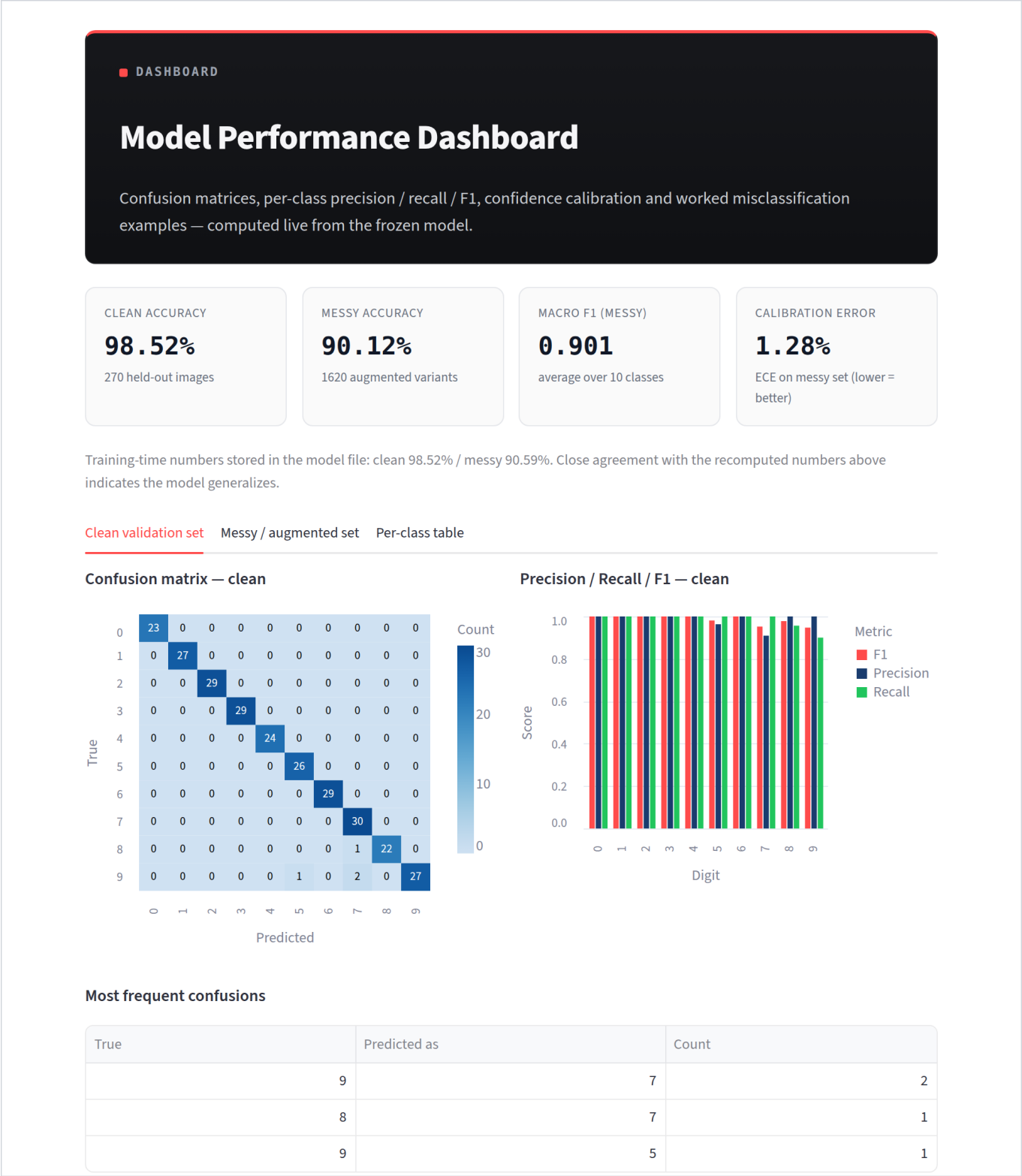
Download CSV

Clear history

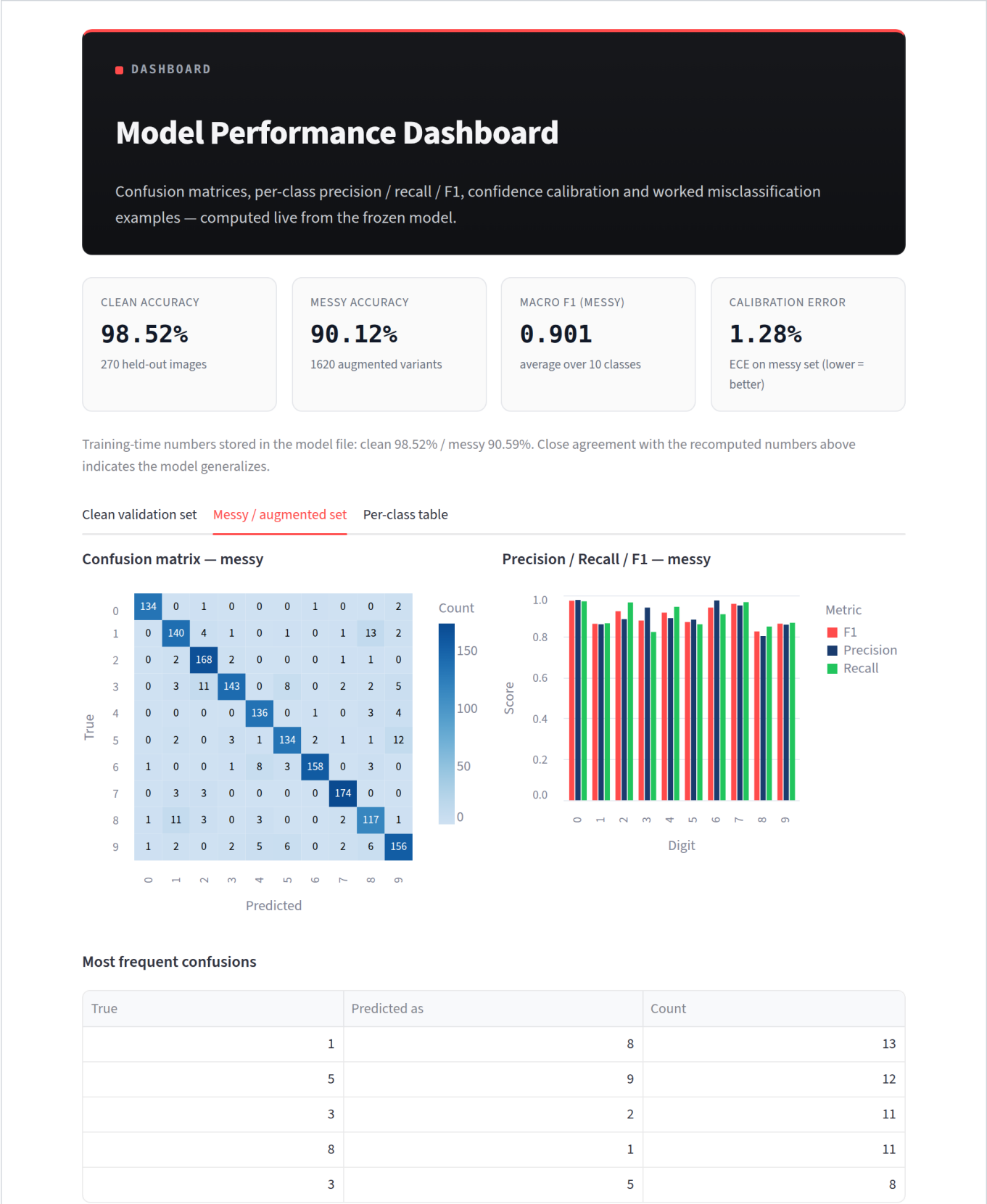
Figure 5: Synthetic test photo of a held-out '2': the original upload, the auto-cropped ink and the 8×8 model input, the result (2 with 66.8 % confidence), and the session history, which after the sample, drawn and uploaded predictions lists exactly three entries, with CSV download and clear buttons.

20 / 31

6.6 Model Dashboard — Clean Validation Set



6.7 Model Dashboard — Messy / Augmented Set



6.8 Model Dashboard — Per-Class Table

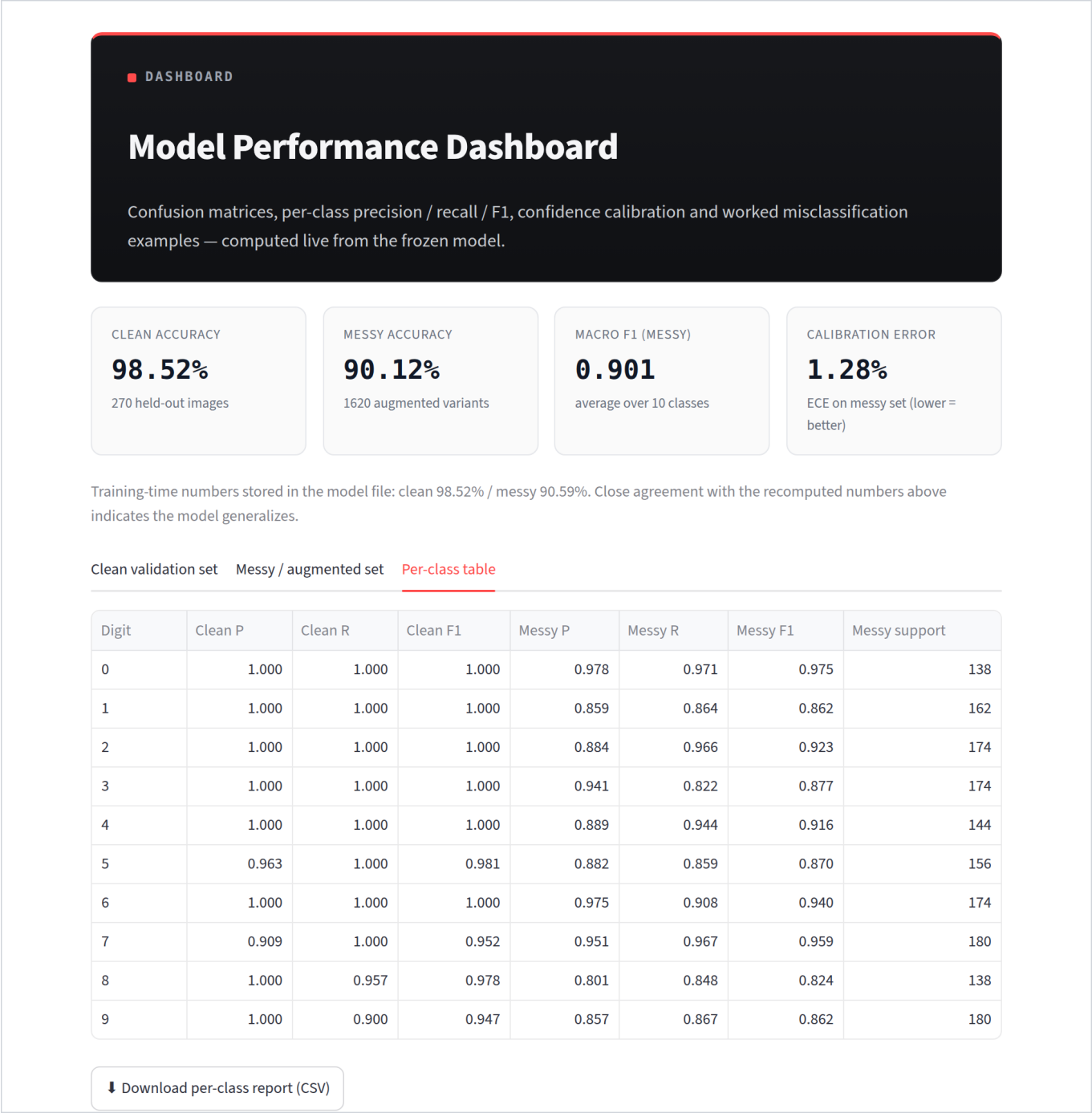


Figure 8: Precision, recall and F1 for each digit on both sets, with the messy-set support, and a button to download the table as CSV.

6.9 Model Dashboard — Calibration and Misclassifications

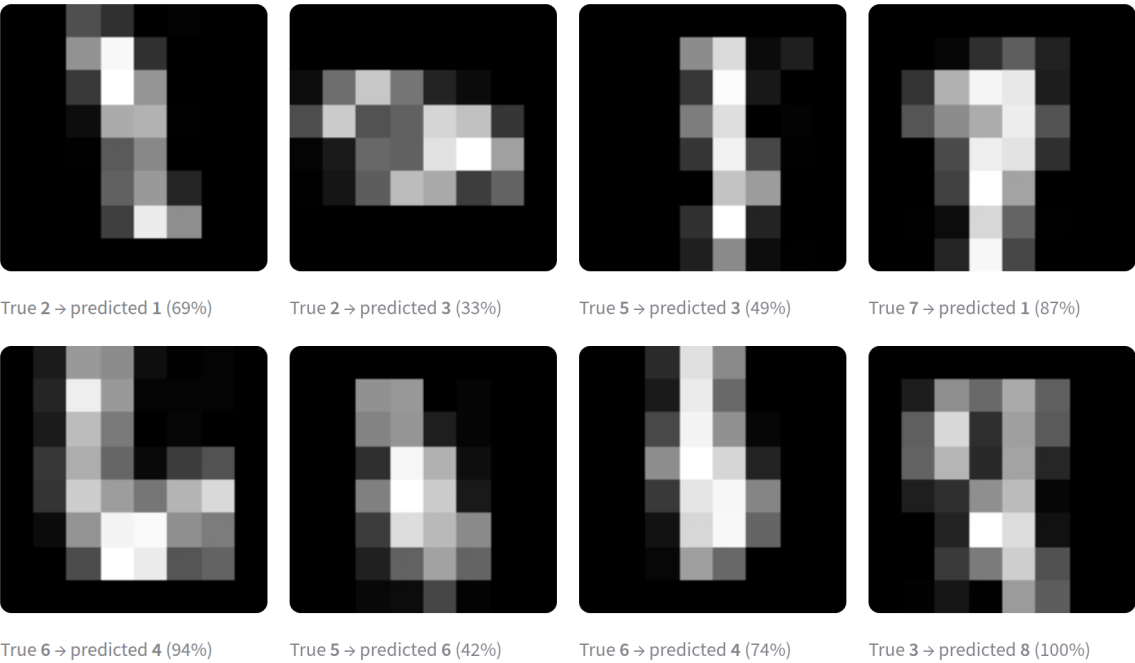
Confidence calibration

Bars should follow the dashed diagonal: when the model says 70% sure, it should be right ~70% of the time.



Misclassification examples

Genuine errors from the messy evaluation set — not cherry-picked successes.



Digit Intelligence Suite · multi-page Streamlit application · CNN trained from scratch in NumPy

Figure 9: Reliability diagram (axes 0–1) and correct-versus-wrong confidence histogram, followed by eight of the twelve saved misclassification examples with true label, predicted label and confidence.

6.10 Dataset Explorer — Class Balance and Mean Digits



6.11 Dataset Explorer — Raw Browser and Augmentation Preview

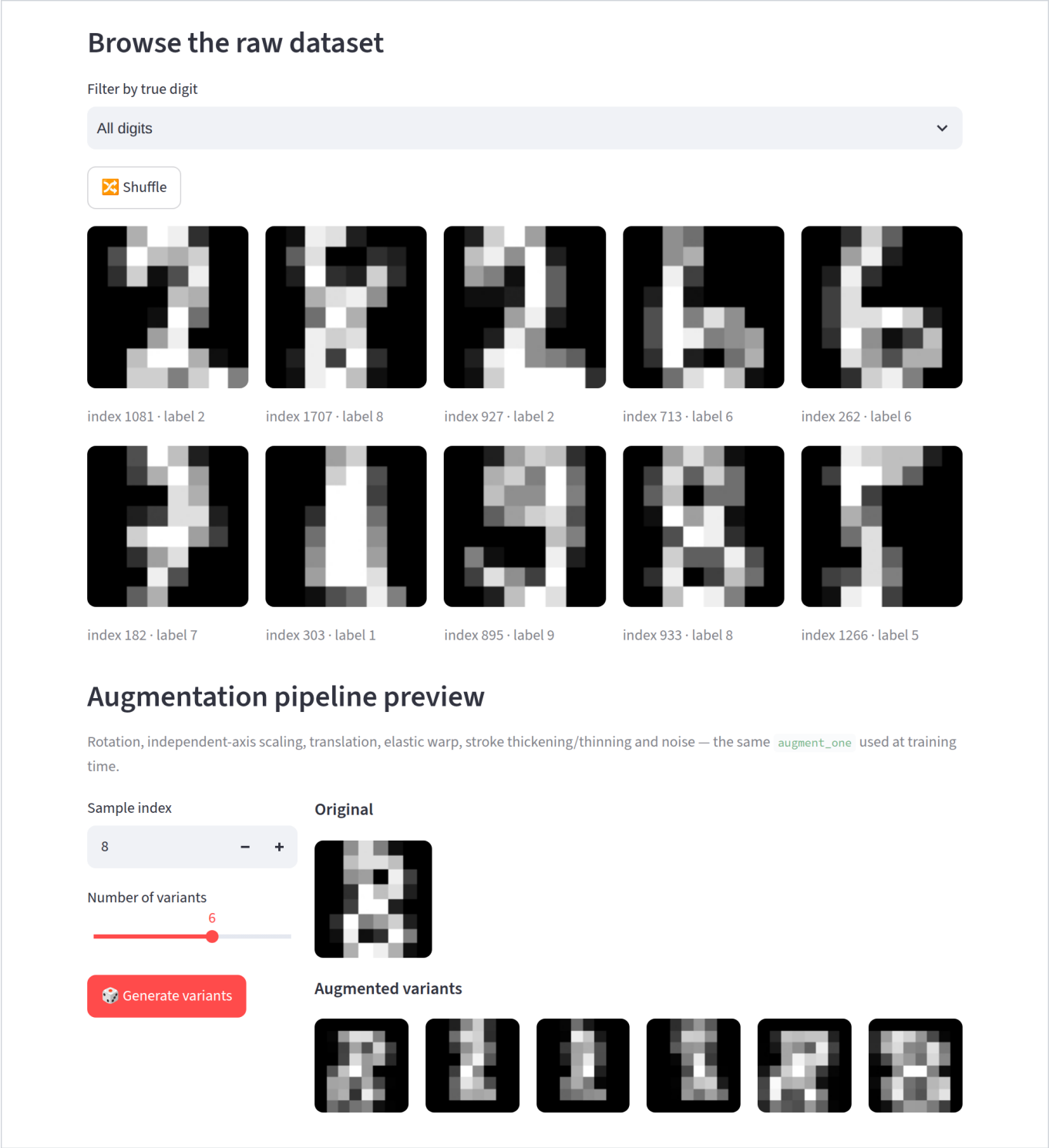


Figure 11: Raw-dataset browser with digit filter and shuffle, and the augmentation preview showing an original '8' (sample index 8) next to six freshly generated variants.

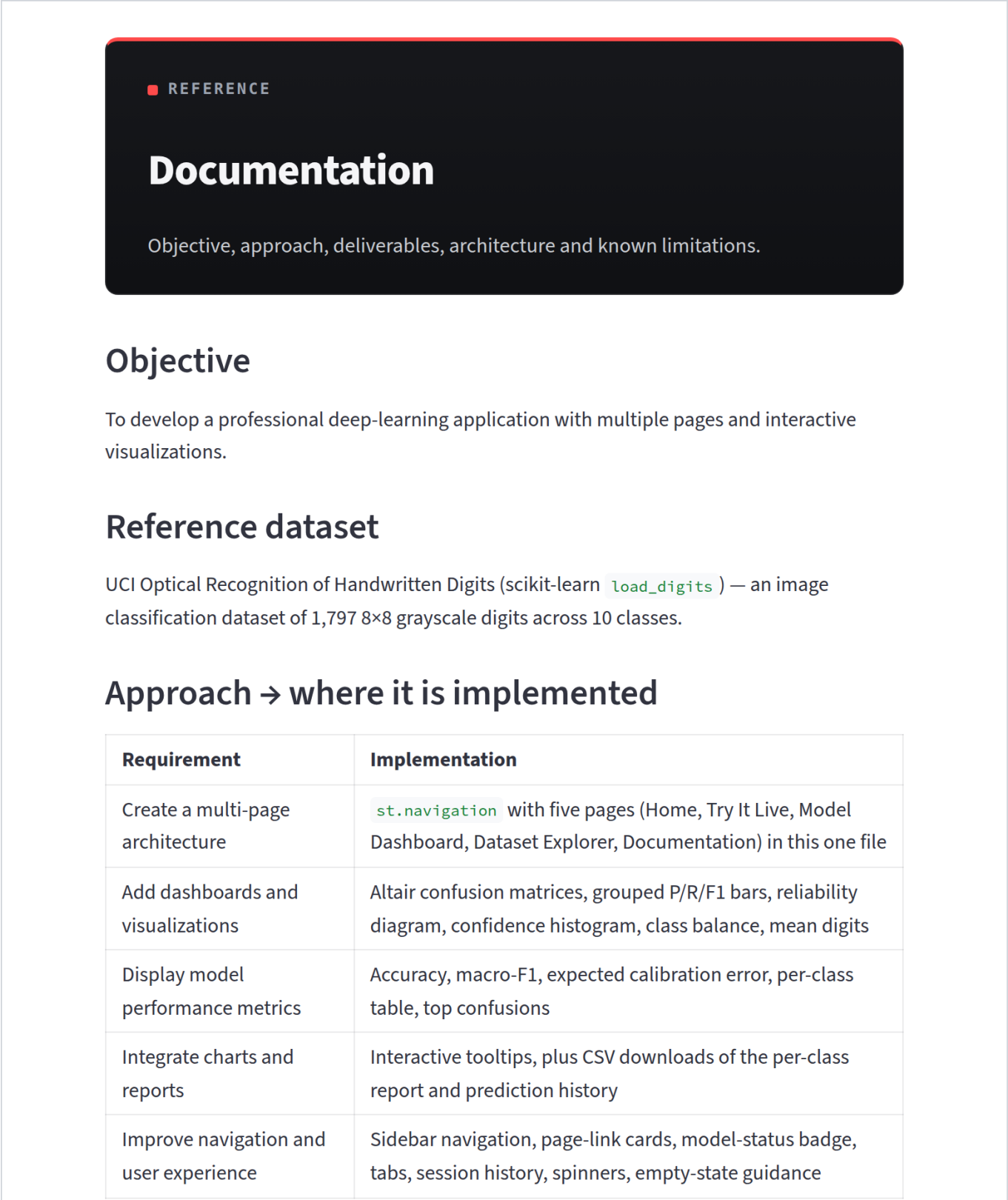


Figure 12: In-app Documentation page (captured in a narrower window): objective, reference dataset and the table mapping each requirement to its implementation.

### Deliverables

- **Multi-page Streamlit application** — `app.py`
- **Dashboard visualizations** — *Model Dashboard* and *Dataset Explorer* pages
- **Project documentation** — this page

### Methodology

A small CNN (Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Dense → ReLU → Dense → Softmax) trained from scratch in NumPy with hand-derived backpropagation. The identical crop-to-ink, independent-axis scale and center-by-mass pipeline is used for training augmentation and for every live input, so training and inference never drift apart. Live predictions average 11 test-time-augmentation variants ( $\pm 1$ px shifts, thicker/thinner strokes).

### Project files

| File                              | Role                               |
|-----------------------------------|------------------------------------|
| <code>app.py</code>               | The entire application (this file) |
| <code>cnn_digits_model.pkl</code> | Trained weights + metadata         |
| <code>requirements.txt</code>     | Python dependencies                |

### Known limitations

- **Small, low-resolution data** — ~1,800 images at 8×8 is not production-grade.
- **Extreme proportions** — very tall/wide digits become ambiguous when compressed to 8×8.
- **Stylistic mismatch** — ornate or heavily cursive digits differ from the plain training style.
- **Evaluation recency** — metrics are computed from the model file currently loaded and cached; restart the app after replacing `cnn_digits_model.pkl`.

Figure 13: In-app Documentation page: deliverables, methodology, project files and known limitations.

## 7. Results

Evaluation protocol: the validation split is the last 15 % (270 images) of a seed-42 permutation of the dataset. The clean pass runs these images through the standard preprocessing only. The messy pass generates 6 augmented variants of each image (1,620 samples, random seed 7) with the augmentation recipe of Section 4.4.

| Metric                                                                    | Value                                         |
|---------------------------------------------------------------------------|-----------------------------------------------|
| Clean validation accuracy (recomputed)                                    | 98.52% (266 of 270)                           |
| Messy/augmented validation accuracy (recomputed, 1,620 fresh variants)    | 90.12% (1,460 of 1,620)                       |
| Clean accuracy recorded at training time (cnn_digits_model.pkl)           | 98.52%                                        |
| Messy/augmented accuracy recorded at training time (cnn_digits_model.pkl) | 90.59%                                        |
| Clean set macro-average F1                                                | 0.986                                         |
| Messy set macro-average F1                                                | 0.901                                         |
| Expected calibration error (messy set, 10 bins)                           | 1.28%                                         |
| Mean confidence of correct / wrong predictions (messy set)                | 93.5% / 67.4%                                 |
| Predictions at $\geq 80$ % confidence (messy set)                         | 1,331 of 1,620 (82.2%); 96.4% of them correct |
| Held-out validation images / augmented variants per image                 | 270 / 6                                       |
| Most common confusions (messy set, true→pred (count))                     | 1→8 (13), 5→9 (12), 3→2 (11), 8→1 (11)        |
| Trainable parameters                                                      | 7,754                                         |
| Model fingerprint (weights hash)                                          | 94e8348e89                                    |

### 7.1 Per-class precision, recall and F1

| Digit        | Clean P | Clean R | Clean F1     | Messy P | Messy R | Messy F1     | Messy support |
|--------------|---------|---------|--------------|---------|---------|--------------|---------------|
| 0            | 1.000   | 1.000   | 1.000        | 0.978   | 0.971   | 0.975        | 138           |
| 1            | 1.000   | 1.000   | 1.000        | 0.859   | 0.864   | 0.862        | 162           |
| 2            | 1.000   | 1.000   | 1.000        | 0.884   | 0.966   | 0.923        | 174           |
| 3            | 1.000   | 1.000   | 1.000        | 0.941   | 0.822   | 0.877        | 174           |
| 4            | 1.000   | 1.000   | 1.000        | 0.889   | 0.944   | 0.916        | 144           |
| 5            | 0.963   | 1.000   | 0.981        | 0.882   | 0.859   | 0.870        | 156           |
| 6            | 1.000   | 1.000   | 1.000        | 0.975   | 0.908   | 0.940        | 174           |
| 7            | 0.909   | 1.000   | 0.952        | 0.951   | 0.967   | 0.959        | 180           |
| 8            | 1.000   | 0.957   | 0.978        | 0.801   | 0.848   | 0.824        | 138           |
| 9            | 1.000   | 0.900   | 0.947        | 0.857   | 0.867   | 0.862        | 180           |
| <b>Macro</b> | 0.987   | 0.986   | <b>0.986</b> | 0.902   | 0.902   | <b>0.901</b> | 1,620         |

## 7.2 Most frequent confusions

**Clean set** (4 errors in total)

| True | Predicted as | Count |
|------|--------------|-------|
| 9    | 7            | 2     |
| 8    | 7            | 1     |
| 9    | 5            | 1     |

**Messy set** (160 errors in total; top 5 shown)

| True | Predicted as | Count |
|------|--------------|-------|
| 1    | 8            | 13    |
| 5    | 9            | 12    |
| 3    | 2            | 11    |
| 8    | 1            | 11    |
| 3    | 5            | 8     |

## 7.3 Reliability data (messy set)

| Mean confidence | Actual accuracy | Samples | Gap (confidence – accuracy) |
|-----------------|-----------------|---------|-----------------------------|
| 26.8%           | 0.0%            | 3       | +26.8 pts                   |
| 36.2%           | 30.0%           | 10      | +6.2 pts                    |
| 45.7%           | 34.3%           | 35      | +11.4 pts                   |
| 54.7%           | 60.0%           | 65      | -5.3 pts                    |
| 64.9%           | 63.1%           | 84      | +1.8 pts                    |
| 75.3%           | 76.1%           | 92      | -0.8 pts                    |
| 85.5%           | 80.9%           | 136     | +4.6 pts                    |
| 98.4%           | 98.2%           | 1,195   | +0.3 pts                    |

Only the populated bins are listed (8 of 10). The three lowest bins hold 3, 10 and 35 samples and are therefore noisy.

## 8. Observations

- The dashboard's recomputed clean accuracy matches the training-time figure exactly (98.52%). The messy accuracy is 90.12% against 90.59% recorded in the model file, a difference of 0.47 points. The augmentation draw is seeded, but results can shift by a few tenths of a point between environments (an earlier run of the same evaluation recorded 89.94%).
- The clean set is almost solved: only 4 of 270 images are wrong (9 → 7 twice, 8 → 7 and 9 → 5 once each) and every class has an F1 of at least 0.947.
- The messy set exposes the weak classes. The lowest F1 scores are for digit 8 (0.824), digit 1 (0.862) and digit 9 (0.862); the highest is digit 0 (0.975). The most frequent confusions are 1 → 8 (13), 5 → 9 (12), 3 → 2 (11) and 8 → 1 (11). The 1/8 pair appearing in both directions is plausible given the extreme stretch and squish distortions, which can make an 8 look like a thin vertical stroke at 8×8 resolution.
- Calibration is good overall: the expected calibration error is 1.28%. 1,195 of 1,620 predictions fall in the top bin, where mean confidence is 98.4% and accuracy is 98.2%. The model is somewhat over-confident in the middle bins (for example 45.7% stated against 34.3% actual) and slightly under-confident in the 54.7% bin, but those bins hold few samples.
- Confidence is informative but not a guarantee. Correct predictions average 93.5% confidence and wrong ones 67.4%, and predictions at 80% confidence or higher are right 96.4% of the time. Even so, 22 of the 160 errors were made at 90% confidence or higher and 7 at 99% or higher; the misclassification gallery includes a 3 read as 8 at 100%.
- The live examples are single demonstrations, not accuracy estimates. The sample (6 at 62.4%), the hand-drawn '3' (70.3%) and the synthetic photo of a '2' (66.8%) were all classified correctly with moderate confidence. Averaging 11 test-time variants tends to lower the top probability when the variants disagree, so moderate confidence on off-distribution input is expected.
- The drawing canvas loaded and worked with Streamlit 1.65.0 and streamlit-drawable-canvas 0.13.0. The pixel-grid fallback for incompatible environments is in the code but was not triggered in this run.

## 9. Known Limitations

- **Small, low-resolution data** — roughly 1,800 base images at 8×8 resolution is a small, simple dataset, not a production-grade one.
- **Extreme proportions** — a digit drawn far taller than it is wide (or vice versa) can become genuinely ambiguous once compressed to 8×8, an information-loss limit of the format rather than a bug.
- **Stylistic mismatches** — the training data uses plain, single-stroke digits, so ornate or heavily cursive strokes can look unlike anything the model has seen.
- **The messy score is not independent of training** — it uses the same augmentation recipe as training, so it measures robustness to those particular distortions, not to unseen handwriting from new writers.
- **Held-out split is reconstructed** — the validation split is rebuilt in `app.py` from seed 42. The clean accuracy equals the recorded training-time value, which is consistent with an identical split, but the training script is not part of this submission, so this cannot be confirmed from the submitted files alone.
- **Noisy calibration at the low end** — the lowest-confidence bins contain very few samples (3, 10 and 35), so their reliability estimates are unstable.
- **Evaluation recency** — metrics are computed from the model file loaded when the server starts and then cached. Restart the app after replacing `cnn_digits_model.pkl`.
- **Environment sensitivity** — the app needs Streamlit 1.53 or newer, and the drawing canvas is a third-party component that can break with other Streamlit versions.

## 10. Conclusion

The Digit Intelligence Suite turns a NumPy-only CNN digit classifier into a complete multi-page Streamlit application: a live classifier with three input modes, a dashboard that recomputes every metric from the frozen weights (clean accuracy 98.52%, messy accuracy 90.12%, macro-F1 0.901, calibration error 1.28%), a dataset explorer with a live augmentation preview, and in-app documentation, all sharing one model and one design system in a single self-contained file. Every number in the dashboard was computed by running the real, frozen model, and every screenshot in this report was captured from the actual running application. The report is candid about what the numbers do and do not show: the messy score measures robustness to the training distortions, calibration is good but not perfect, and the model remains limited by the small 8×8 dataset.